INDEPENDENT PROTOTYPE / ORACLE LINUX 9

HeatWave Audit Center

User manual

From a source-table change to traceable BEFORE and AFTER history.

A prototype for evaluation, demonstration and rehearsal.
This is not an Oracle product and is not supported or endorsed by Oracle.

Overview and architecture · Installation and setup · Screen and action reference · Operations · Scaling, updates and cleanup

28 October 2026
Implementation review: 28 October 2026
Screenshots use isolated, illustrative sample data.

Contents

Overview and design

1. Background and prototype status 4
2. Challenges and the implemented approach 5
3. Deployment architecture 6
4. Network access and security boundaries 7
5. Data flow, fingerprints and evidence 9

Setup and installation

6. Installation prerequisites 10
7. Install and verify the OL9 services 10
8. IAM and database access checklist 11
9. First working flow: setup order 12

Using the application

10. Sign in and password rotation 13
11. Navigation, tables and notifications 15
12. Dashboard 17
13. Configuration: HeatWave connections 18
14. Edit registrations without breaking continuity 21
15. Configuration: register a table mapping 23

Configuration and capture design

16. Source scopes and target layouts 25
17. Streaming buffers, size limits and request control 26

Using the application

18. Runtime settings and fingerprint-key setup 30
19. Source validation and full row images 31
20. OCI resources: common controls 33
21. Native OCI Streaming: pools and streams 34
22. Queue registrations and topic management 35
23. Container Registry and image builds 37
24. Container Instances and workload settings 39

25. Vault and password references 41

26. HeatWave DB resource controls 43

27. Embedded repository page 45

28. Control Center: flow row and status 46

29. Control Center: action conditions 47

30. Capture start, GTID sets and purged history 50

31. Apply a reviewed capture position 51

32. Message Explorer 52

33. Audit Center: all databases overview 53

34. Audit Center: table history and lookup 54

35. Before/after images and operation meanings 56

36. Charts, intervals and downloads 57

37. Schema changes and security auditing 58

Operation and maintenance

38. Daily operations and a controlled verification 60

39. Summary and handover checklist 60

Appendices

Appendix A. Scaling and massive change volume 62

Appendix B. Changing configuration safely 63

Appendix C. Update the application and workloads 64

Appendix D. One-action eligible payload cleanup 65

Appendix E. Target data layout and retrieval 66

Appendix F. Troubleshooting by observed stage 67

Appendix G. Security, backup and recovery boundaries 69

Appendix H. Glossary and further reading 70

Appendix I. DDL-driven row changes and audit coverage 70

Appendix J. HeatWave ETag issue and process errors 72

Appendix K. Same-compartment resource limitation 73

OVERVIEW AND DESIGN

1. Background and prototype status

HeatWave Audit Center is an independent prototype for exploring row-change auditing with OCI MySQL HeatWave. It brings connection setup, table registration, capture controls and audit-history exploration into one web application on an Oracle Linux 9 Compute VM.

This is a prototype for evaluation, demonstration and rehearsal. It is not an Oracle product and is not supported or endorsed by Oracle. Oracle service names and architecture symbols describe the integration; they do not imply Oracle authorship, certification or a production guarantee.

The aim is to retain evidence of changes to registered tables: what a row contained before and after an INSERT, UPDATE or DELETE; which source position and queue delivery produced it; and how activity changes over time. The target stores audit history rather than maintaining a current-row replica.

Audience and scope

Operators use the Dashboard, Control Center, Message Explorer and Audit Center. Application administrators also manage configuration, resources, capture positions and retention. An OCI administrator prepares IAM and networking; a database administrator prepares schemas, users and supported source settings.

This manual describes the repository implementation as reviewed on 28 October 2026, including the lifecycle, diagnostic and coverage clarifications. Screenshots show representative sample records, not a live OCI acceptance test. Endpoint names, resource identifiers and CIDRs are examples. Never copy the example identifiers into a real deployment.

What is available

- Native OCI Streaming capture for one table, database wildcards or all-database wildcards; grouped audit targets; complete images when recorded; GTID initialization, replay checks, typed-key lookup and charts.
- Conditional large-event fragmentation, durable reassembly, PUT batching/byte pacing and adaptive GET polling.
- Public schema evidence with available before/after definitions and semantic differences; SQL inspection is used for DDL evidence.
- OCI Container Instances lifecycle and image replacement, same-compartment resource management and explicit partition cleanup.

Important boundaries

User/privilege auditing needs a separately configured database-audit collector; it is not implemented here. DML binlogs do not establish a complete actor or privilege history. Automatic fleet scaling, verified archive export/restore, fingerprint-key rotation and a dedicated schema-comparison browser are not implemented. Validate the chosen deployment, recovery and capacity before relying on it.

Row-audit coverage: partition exchange, partition drop/truncate and other DDL can change many rows without individual row-image events, even with ROW/FULL. Schema evidence, fingerprints and event charts cannot establish complete data-change coverage. See Appendix I and audit limitations.

OVERVIEW AND DESIGN

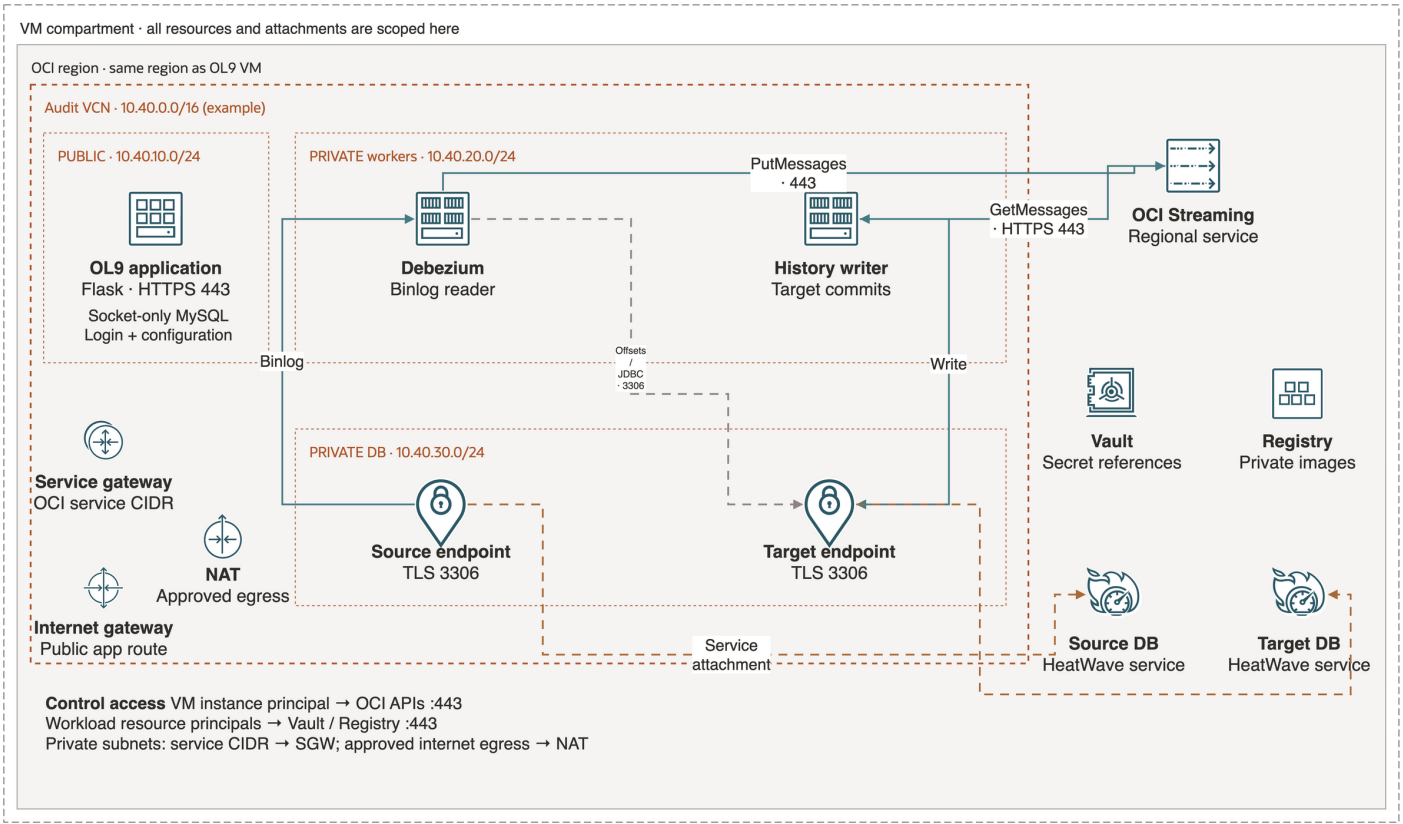
2. Challenges and the implemented approach

Challenge	Implemented approach	Operational boundary
Large source tables	End or Specific GTID avoids copying current rows	Existing unchanged rows have no snapshot baseline
Multiple source tables	One table, database wildcard or all-database wildcard per flow	Stable keys, metadata inventory and scope checks still required
Target organization	Audit tables grouped by flow, source database or source table	Generated tables stay in one selected target schema
Target apply temporarily delayed	Native OCI Streaming retains pending changes	Capture offsets/JDBC history still depend on the target database
Large full-image events	Conditional fragmentation and durable reassembly	Logical/custody, target packet and memory limits still apply
Request throttling	PUT batching/byte pacing and adaptive GET polling	Retries are bounded fallback handling
Recovery and replay	Full offsets, stream identities, deduplication and target delivery state	No global distributed exactly-once transaction

This prototype makes observed changes traceable. It does not establish complete security auditing, reconstruct unavailable pre-capture history, or expand DDL into missing row images. Review the coverage and storage limitations before adoption.

OVERVIEW AND DESIGN

3. Deployment architecture



Native OCI deployment architecture. Illustrative reference.

The OL9 application VM manages registered resources with its instance principal. A private Container Instance runs embedded Debezium Engine and the OCI Java SDK; another runs the Python history writer and OCI Python SDK. Both use resource principals for workload access. Private registry images are digest pinned.

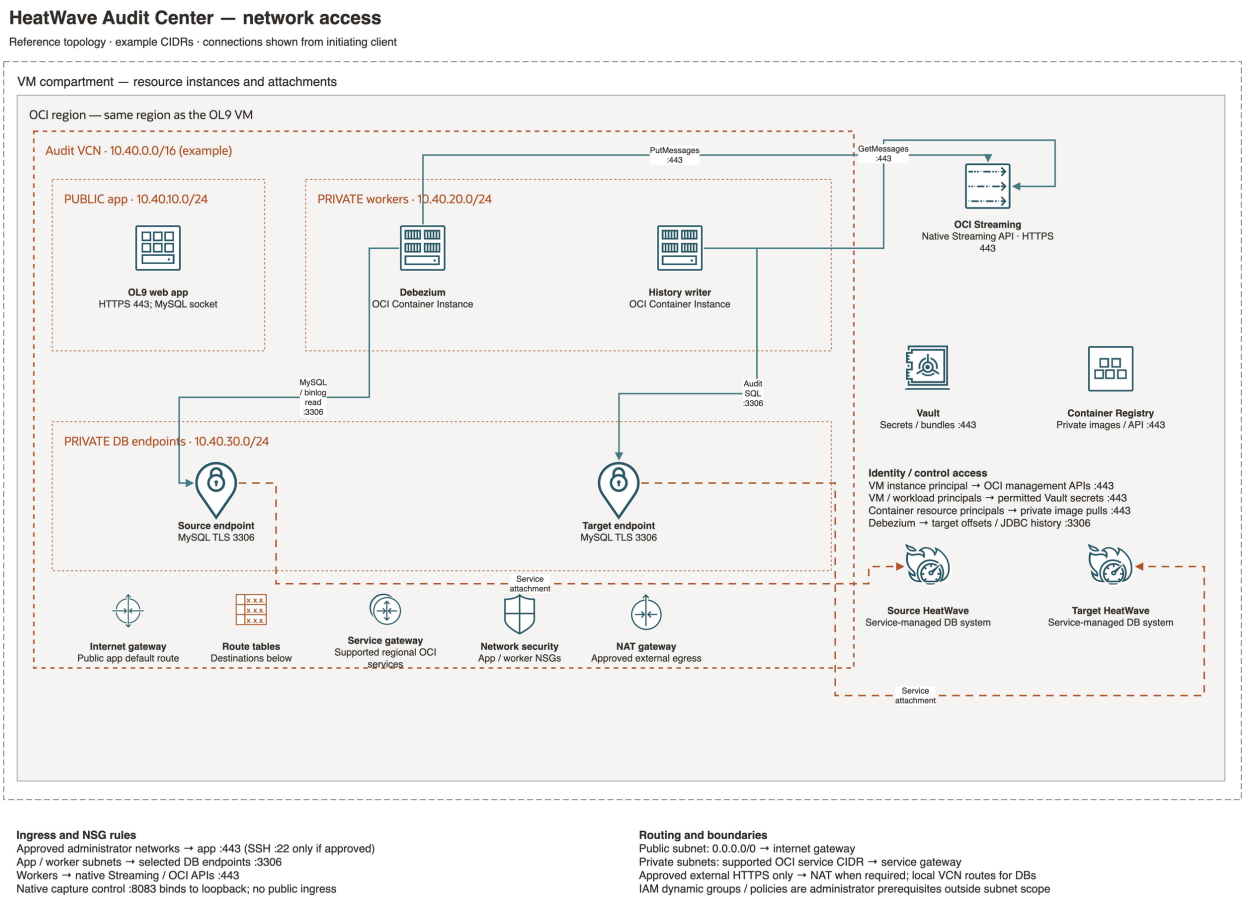
Source and target database endpoints attach to the database subnet. Regional Streaming, Vault, Registry and HeatWave services sit outside the VCN boundary; private attachments are shown separately. CIDRs are illustrative examples.

The target contains audit payloads, deduplication evidence, routes, fragment custody, delivery checkpoints, complete source offsets and private JDBC schema history. Source and writer restarts preserve those namespaces. A full target outage can therefore pause capture as well as apply; source binlog retention remains essential.

Editable deployment architecture

OVERVIEW AND DESIGN

4. Network access and security boundaries



Network access and principal boundaries. Illustrative reference.

Connection	Purpose
Approved browser → application HTTPS 443	Authenticated UI; CSRF protects actions
Application / Debezium → source TLS 3306	Metadata checks, optional row snapshot and replication reads
Application / workloads → target TLS 3306	Audit queries/commits, offsets, JDBC history and recovery state
Workers → native Streaming HTTPS 443	OCI SDK PutMessages and GetMessages
VM / workloads → permitted OCI HTTPS 443	Secret retrieval, registry and scoped service operations
Native capture control on container loopback	Internal engine control; no public ingress

Debezium initiates the source MySQL connection; binlog data flows back on that connection. A snapshot queries source rows directly. The writer connects separately to the target. IAM permits an operation but does not create a network route.

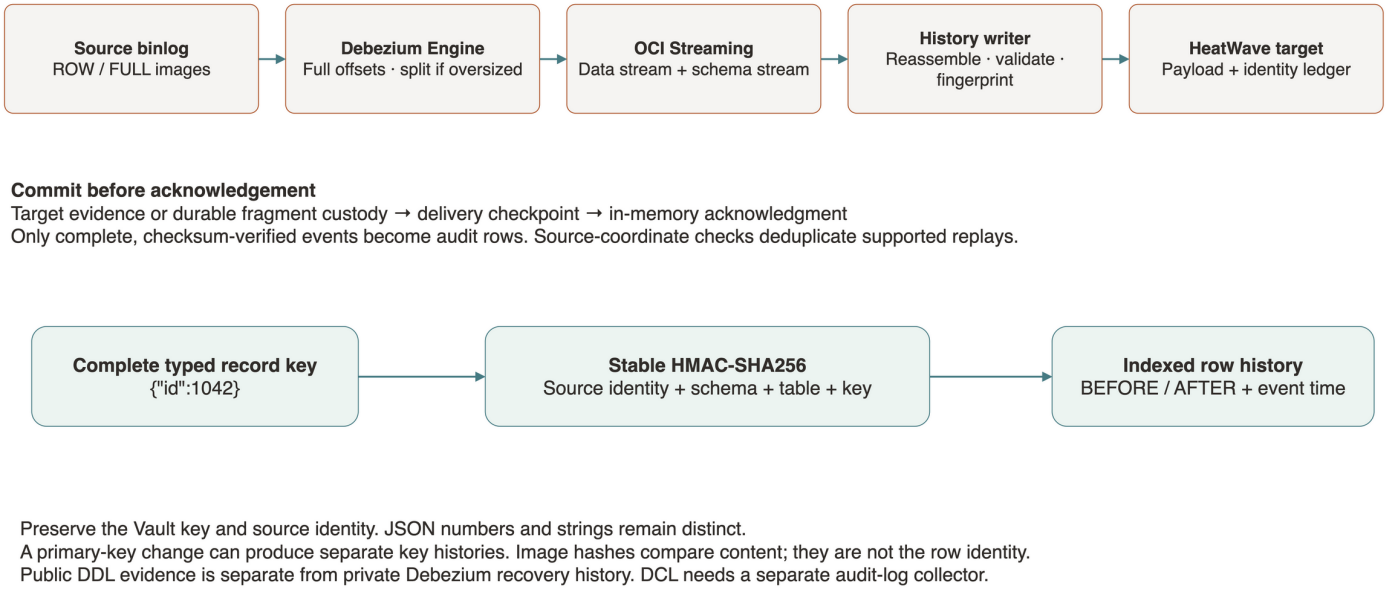
The reference public subnet uses an internet gateway. Private subnet access uses approved regional service-gateway destinations and any separately approved NAT egress. Check endpoint DNS, routes and NSGs together. Private service endpoints need their own attachment/routing review. The app does not alter shared networking.

Editable network reference

OVERVIEW AND DESIGN

5. Data flow, fingerprints and evidence

Capture, commit and retrieve one row history



Capture, delivery and fingerprint lookup. Illustrative reference.

A row change passes through source binlog, embedded Debezium Engine, the data stream and the history writer. Schema events use a separate single-partition stream. Source recovery uses target-backed full offsets and JDBC schema history. Small events remain single messages. Large events are framed and reassembled before normal audit writes; partial fragments are never audit rows.

The writer validates events, computes identities, writes fresh payload/deduplication evidence, and saves target delivery checkpoints before acknowledging in memory. These are recoverable stages, not a single distributed transaction. Durable fragment custody can permit delivery progress before the whole event arrives.

- **Row-key fingerprint:** keyed hash of canonical typed source identity, actual database/table and complete primary key.
- **Before/after fingerprints:** keyed hashes of observed row content.
- **Event/delivery identity:** immutable source coordinates identify supported DML replays; stream coordinates describe deliveries. A GTID alone does not identify each row event.

Canonical JSON preserves types and sorts object keys. `{"id":1042}` differs from `{"id":"1042"}`. Preserve the fingerprint secret and source identity with retained history. Automatic stitching of histories across a primary-key change is not implemented.

Editable workflow

SETUP AND INSTALLATION

6. Installation prerequisites

Deployment is supported on **Oracle Linux 9 only**. Use the non-root application owner with sudo and an architecture-compatible Python 3.12+ environment. The installer uses app-managed MySQL 9.x, Gunicorn/systemd, rootless Podman, Java and Maven dependencies.

Prepare before installation

1. Choose an OL9 Compute VM and the one compartment that will contain this deployment's resources and network attachments. Confirm region, capacity and service availability.
2. Have the OCI administrator configure the VM instance-principal group, container resource-principal group and compartment-scoped permissions. Prepare VCN/subnets, routes, DNS and NSGs outside the application.
3. Have the DBA prepare source and target schemas/accounts. The source needs replication/full-image settings and a stable table primary key. The target schema needs audit-table/state preparation rights.
4. Create permitted Vault password secrets and identify an existing encryption key for a dedicated fingerprint secret. A database password and fingerprint key serve different purposes.
5. Obtain the official MySQL 9.x Linux generic tarball URL for the VM architecture and its independently verified SHA256.
6. Clone the repository on the VM and create private deployment JSON locally from `examples/deployment.example.json`. Fill real values on the VM; do not publish them or copy another installation's private identity.

Keep the private configuration directory mode 0700 and JSON mode 0600. The selected `compartment_id` and `region` must match the VM. Source endpoint login testing does not verify which OCI DB system owns an IP.

Preconditions that can block setup

A missing or invalid configuration, non-OL9 OS, wrong interpreter, invalid MySQL checksum, unsupported server major version or insufficient sudo/network access prevents completion. Resolve the cause before rerunning. Setup is rerunnable, but is not a MySQL major-version migration tool.

Reference: setup and operations and IAM/database access.

SETUP AND INSTALLATION

7. Install and verify the OL9 services

For a fresh VM checkout, clone `https://github.com/ivanxma/hw_data_audit_center.git`, enter that checkout and prepare its private deployment JSON as described in section 6.

Run these examples as the application owner from the checkout. Replace placeholders privately; never place passwords on the command line.

```
export AUDIT_CONFIG_FILE=/path/to/private/deployment.json
export AUDIT_MYSQL_TARBALL_URL='https://official-host/path/mysql-9.x-linux.tar.xz'
export AUDIT_MYSQL_TARBALL_SHA256='<verified-sha256>'
./setup.sh
.venv/bin/python scripts/enroll_admin.py
```

Enrollment prompts for an explicit bootstrap password. First login requires changing it and signing in again. Rerunning enrollment does not reset an existing administrator. The manual intentionally provides no reusable bootstrap password.

```
.venv/bin/python scripts/check_setup.py
.venv/bin/python scripts/check_setup.py --oci --transport streaming
sudo systemctl status heatwave-audit-local-mysql.service
sudo systemctl status heatwave-audit-https.service
sudo systemctl status heatwave-audit-image-builder.service
```

Service	What to verify
heatwave-audit-local-mysql	Local Unix socket available; no public MySQL listener
heatwave-audit-https	App running on configured HTTPS port, normally 443
heatwave-audit-image-builder	Builder available before Build & publish

The installer saves the configuration-file selection and runtime paths in private `.runtime.env`. It generates a self-signed certificate initially; for trusted browser access install a certificate matching the intended DNS name. Custom certificate and key paths must be supplied together.

The HTTPS listener binds to `0.0.0.0`; browse the VM's actual address or DNS name. The app's development HTTP launcher defaults to loopback and is not the deployment entry point. Restrict access with host and OCI rules. Read-only access checks do not prove create permissions or end-to-end capture.

Before a setup rerun, back up the repository/configuration and ensure no image build is active. Do not replace a working embedded MySQL binary as an incidental application update.

SETUP AND INSTALLATION

8. IAM and database access checklist

The VM uses its **instance principal** for OCI management. Container Instances use their own **resource principals** for private images, secrets and Streaming data access. The container does not inherit VM permissions. The application does not switch to user API keys when access is denied.

OCI administrator tasks

Use the full IAM policy guide, replacing placeholders with the actual identity-domain/group subjects and VM compartment. Grants depend on enabled features. Typical capabilities include container lifecycle, private repositories, permitted secret metadata/bundles and use of existing network attachments.

`manage stream-family` in the VM compartment covers OCI streams and stream pools. Native produce/consume permissions are needed for the relevant principal. Container principals need permitted produce/consume/read access and secret/repository access separately. A list success proves that read operation only.

App ownership tags guard app actions but are not an IAM isolation boundary. Review dynamic-group membership, especially rules that match every Container Instance in a compartment. The application cannot authorize changes outside its compartment merely because a broader IAM policy exists.

MySQL account responsibilities

Account	Required purpose
Dedicated source connector	SELECT, SHOW DATABASES, REPLICATION SLAVE, REPLICATION CLIENT, LOCK TABLES; RELOAD or supported FLUSH_TABLES alternative
Target app/writer	SELECT/INSERT/UPDATE/DELETE plus CREATE/ALTER/INDEX on the chosen audit schema for preparation and maintenance
Target Debezium JDBC state	CREATE/SELECT/INSERT/UPDATE/DELETE on its private history and request state; currently reuses the flow's target credential
Local administrator	Embedded repository/socket authentication; separate from source and target accounts

Have the DBA review scoped grants and role activation. The source validation fails closed when it cannot establish required access. Database/schema creation and grants are prerequisites; validation does not apply them. Production account separation and removing preparation rights require a tested migration/runtime design; a separate migrator account is not currently exposed.

Keep safe OCI request IDs for troubleshooting. A 403/404 may indicate membership, policy, wrong resource scope, readiness or regional availability. Retry the same intended principal after correction; do not substitute tenancy-wide grants.

SETUP AND INSTALLATION

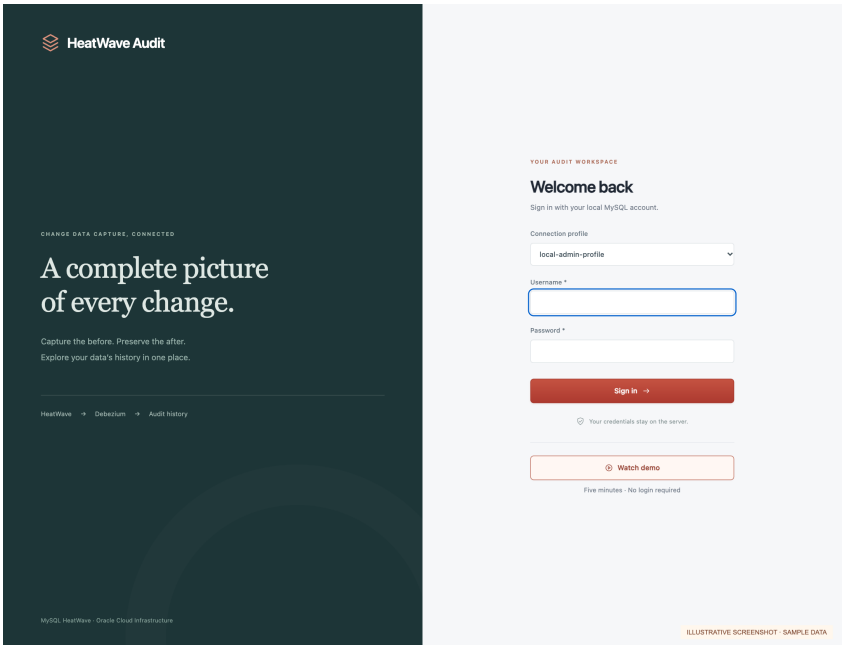
9. First working flow: setup order

1. Verify the OL9 application and embedded repository login. Run setup/access checks.
2. Register source and target connections using Vault password references; test both.
3. Review source ROW/FULL binlogging, GTIDs, retention, stable keys and grants.
4. In OCI resources, select/create the active Streaming pool and data stream. Register the pool connection without a username or messaging token.
5. Build/publish the native Debezium and writer images in the private registry. Select compatible digest-pinned releases and private workload networking.
6. Register a mapping: one table, `database.*` or `*.*`; choose target grouping and End, Start or Specific GTID. End is the default and avoids a row-copy snapshot.
7. Run source checks. Prepare/select the data stream; prepare the single-partition schema stream `audit_<flow-id>`. Deployment also prepares the schema stream automatically.
8. Deploy. Confirm the source task and writer are running, then verify queue arrival and newest target events/checkpoints.
9. Use approved isolated CREATE/UPDATE/DELETE operations to verify typed keys, full images, actual source table labels and replay behavior.

Keep a reviewed record of stream IDs, source lineage, offsets, JDBC history, fingerprint secret and delivery state. Container ACTIVE alone does not prove capture.

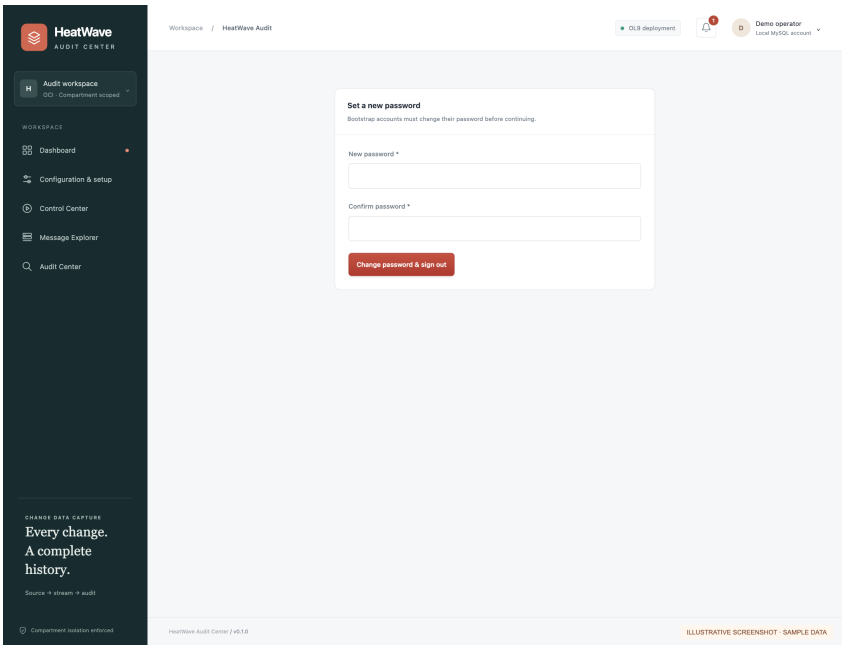
USING THE APPLICATION

10. Sign in and password rotation



Sign-in page. Illustrative screenshot using the current templates and isolated sample data.

Field / action	Meaning and when to use it
Connection profile	Select local-admin-profile, the embedded MySQL authentication profile
Username	Your local application/MySQL account; initial administrator enrollment creates localadmin
Password	Enter privately; it is not prefilled or shown in the browser
Sign in	Requires valid credentials and profile; a bootstrap account is redirected to rotation



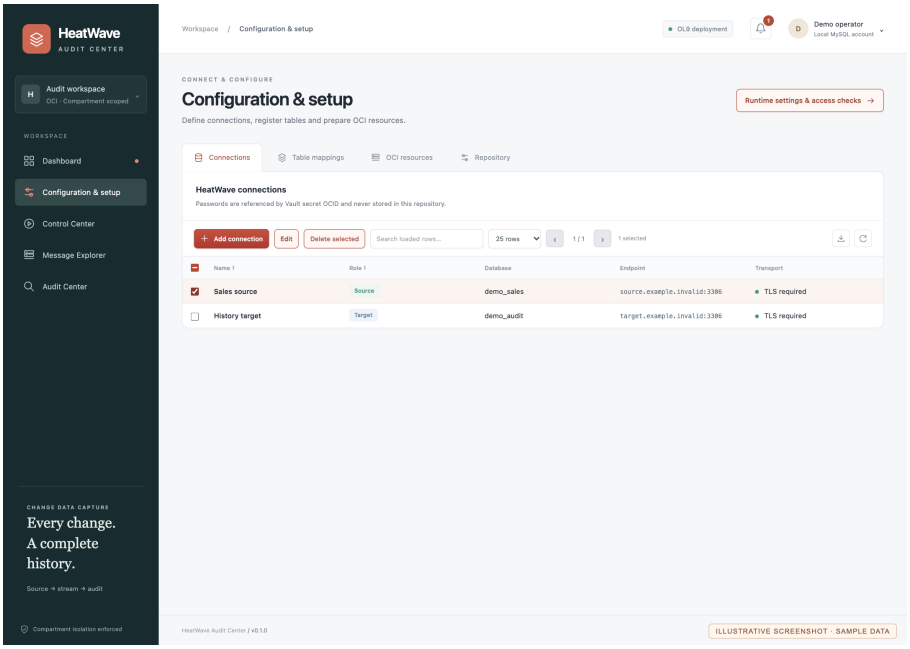
Password-change page. Illustrative screenshot using the current templates and isolated sample data.

Enter the same new password in **New password** and **Confirm password**. The UI requires at least 14 characters. **Change password & sign out** rotates the password, clears account sessions and requires a new login. The profile/user menu also provides Change password after login.

The authenticated shell uses server-side session state; the browser receives an opaque session identity. The displayed session expiry is 60 minutes. When the session expires, sign in again before attempting an action. Configuration and lifecycle operations require administrator authorization; a read-only page does not grant mutation rights.

USING THE APPLICATION

11. Navigation, tables and notifications

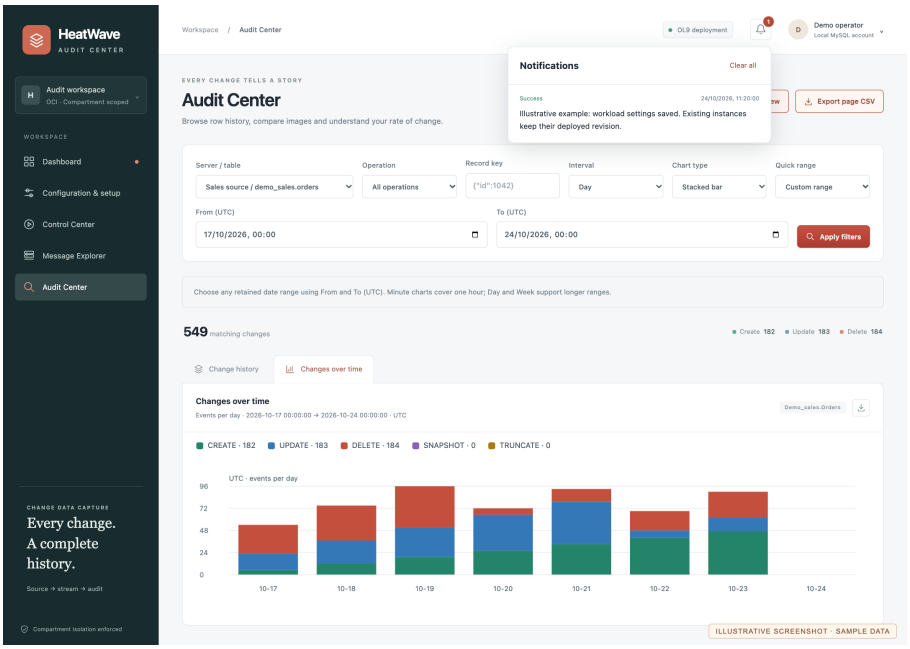


Single-row selection and the shared table toolbar. Illustrative screenshot using the current templates and isolated sample data.

The left navigation opens Dashboard, Configuration & setup, Control Center, Message Explorer and Audit Center. Configuration is administrator-only. The top-right account menu shows profile/version, Change password and Sign out.

Shared table behavior

- Use row checkboxes for selection. **Edit** requires exactly one registration; flow lifecycle actions also require exactly one flow.
- Search filters loaded rows, not the entire database. The page-size selector and arrows page loaded results; Audit Center also has server result pages.
- Click a column heading to sort loaded rows. Drag column boundaries to resize, drag headers to rearrange and use the reset-layout icon to restore the table layout.
- Click a field value, or focus it and press Enter, to open its full value. Close or Escape returns to the table. Links, selection boxes and action controls retain their own behavior.
- The right-side download icon exports loaded/selected table rows as CSV as indicated by the toolbar. A loaded-page export is not a full audit archive.



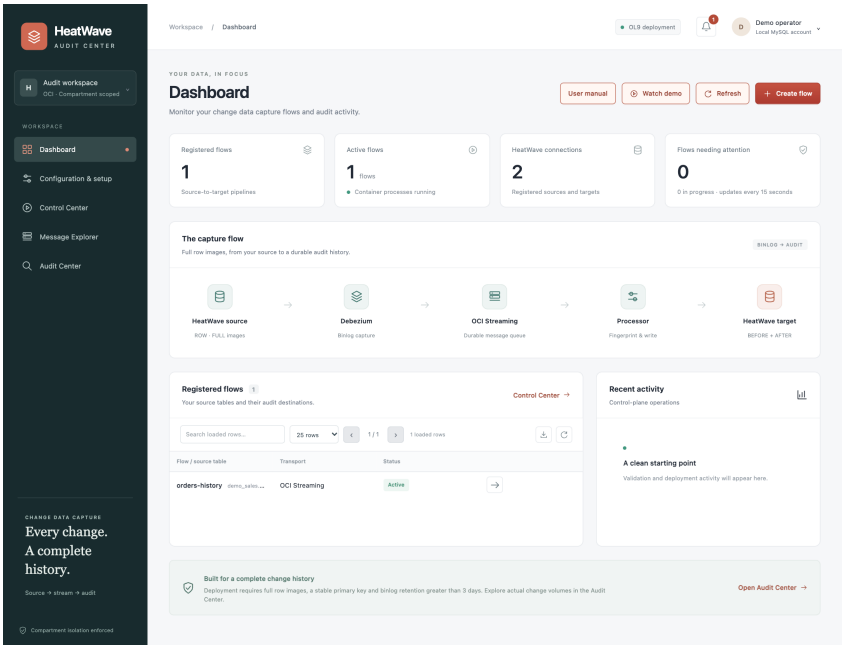
Top-right notification bell and message history. Illustrative screenshot using the current templates and isolated sample data.

The bell opens recent messages for the current user/session. **Clear all** clears that list when messages exist. The × on a top message bar dismisses only that bar; the previous message remains available in Notifications. Build completion/failure messages can also appear there. Read the underlying operation status after a timeout; notification clearing does not undo an operation.

Enabled buttons can still be rejected by server-side scope, ownership, state and readiness checks. A grey disabled button needs the required selection/state or permission; use the on-screen guidance to identify which.

USING THE APPLICATION

12. Dashboard



Dashboard with registered flows and recent control-plane activity. Illustrative screenshot using the current templates and isolated sample data.

Area / action	Interpretation
Registered flows	Number of registered source-table mappings
Active flows	Current process/task health summary; it is not a count of successful deliveries
HeatWave connections	Registered source and target connection count
Flows needing attention	Draft, pending, failed/unavailable or otherwise incomplete flow health
Capture flow strip	Conceptual path from source to target; it is not live throughput telemetry
Registered flows table	Open history with the right arrow; Control Center opens lifecycle controls
Recent activity	Control-plane actions/outcomes and actor/time, not every source SQL statement
Refresh	Reload the dashboard; flow health also refreshes periodically
Create flow	Administrator shortcut to table registration

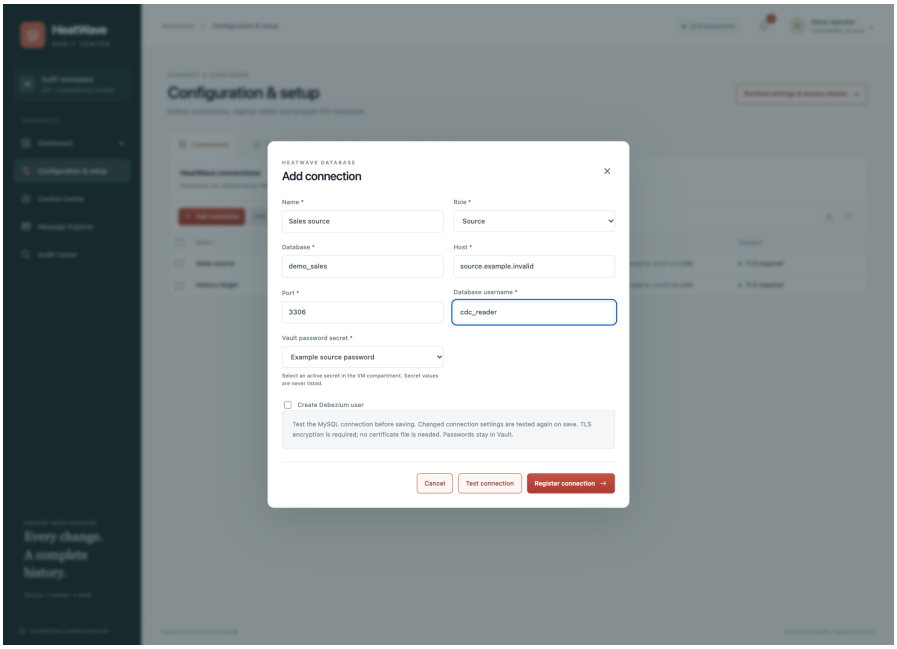
Use this page for a first health overview. If a flow needs attention, open Control Center and inspect both workload tips. A green process state does not establish that data has arrived. Use Message Explorer and Audit Center for evidence, and workload logs/checkpoints for diagnosis.

Watch demo opens the signed-in `/demo/` playback page. Play or pause the narrated video in the browser, use native seek/full-screen controls, enable English captions or select a chapter. The page does not require an MP4 download.

User manual opens the deployed HTML guide. Its downloads include the PDF and PowerPoint overview. Watch demo opens the signed-in video player with narration, captions and chapters.

USING THE APPLICATION

13. Configuration: HeatWave connections



Add connection dialog. Illustrative screenshot using the current templates and isolated sample data.

Open **Configuration & setup → Connections → Add connection**.

Field	Meaning / condition
Name	Human-readable registration name; up to 100 characters
Role	Source for binlog extraction or Target for audit history
Database	Accessible database/schema name; DBA creates it beforehand
Host	Supplied reachable DB endpoint or IP; no DB system OCID lookup
Port	MySQL TCP port; default 3306, valid range 1–65535
Database username	Dedicated source or target account with the required grants
Vault password secret	Required dropdown of ACTIVE secrets by name in the VM compartment; select the secret containing this account's password

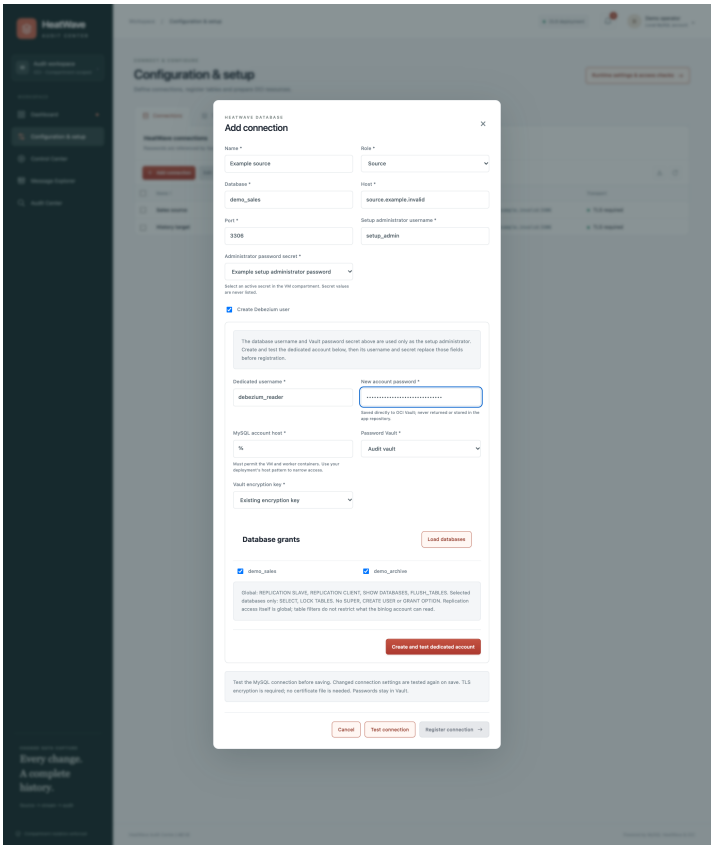
Test connection requires valid form inputs and performs a real MySQL login using the selected endpoint and Vault reference. A successful test checks connectivity/TLS/login, not binlog readiness; run Validate source later. **Register connection** saves after server validation and connection testing. Changed settings are tested again on save. **Cancel / ×** closes without saving.

MySQL Connector/Python is used. TLS defaults to REQUIRED: encryption is required but a per-connection certificate/CA file is not requested. Optional global verified trust belongs in Runtime settings. The Vault secret is a password reference, not a certificate path.

The list's **Delete selected** removes allowed unused registrations, not the underlying HeatWave DB system. Referenced connections cannot be removed. Use **Edit** with one selected row; deployed/reference restrictions are explained next.

Create a dedicated Debezium or history writer account

For a new connection, optionally tick **Create Debezium user** (Source) or **Create history writer user** (Target). The original username/secret inputs become temporary setup-administrator inputs. They are used to create the dedicated account, not registered as its runtime credentials.



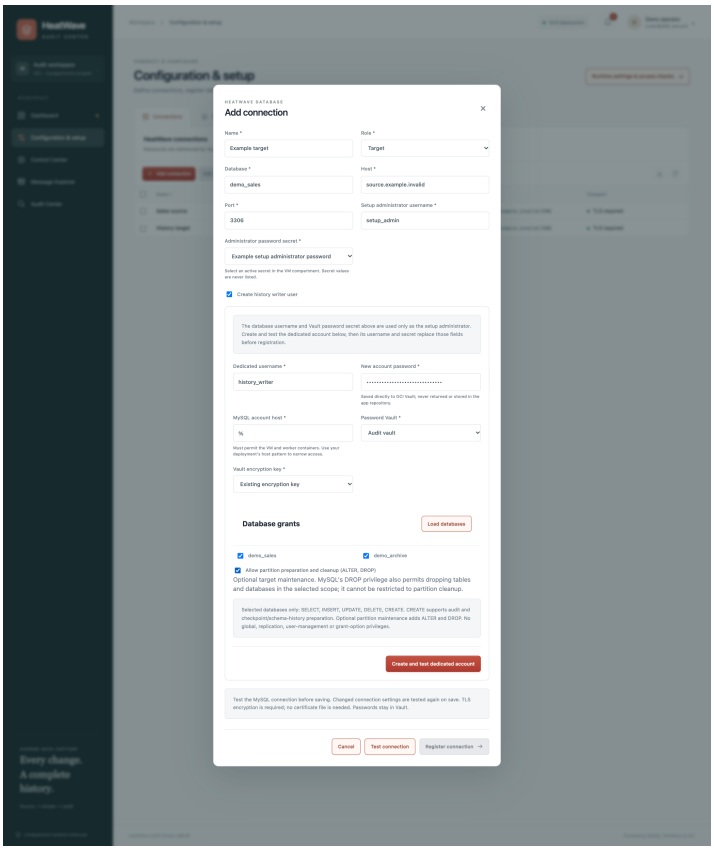
Illustrative source setup with two selected databases; the entered password remains masked.

Field / action	Meaning and condition
Setup administrator username / secret	Existing administrator and a password secret selected from the same-compartment dropdown, used only during setup; requires CREATE USER and grantable planned privileges
Dedicated username	New account, different from the administrator; existing unmanaged users are never reset
New account password	12–256 characters and compliant with the server policy; sent directly to MySQL/Vault, never returned in the response or stored in the app repository
MySQL account host	Client-host pattern permitting the VM and worker containers; narrow the default % when appropriate

Field / action	Meaning and condition
Password Vault / encryption key	ACTIVE Vault and enabled AES key in the VM compartment; new credential secret is created there
Load databases	Test administrator access and fetch live database names
Database checkboxes	Multi-select the needed databases, including the connection default; up to 50
Allow partition preparation and cleanup	Target only, opt-in ALTER/DROP grants; DROP also permits dropping tables/databases in the selected scope
Create and test dedicated account	Store the credential, create the TLS-required user, apply/verify grants, then test its login
Register connection	Enabled after successful setup; re-tests and registers the populated dedicated username/secret

Source grants are global REPLICATION SLAVE, REPLICATION CLIENT, SHOW DATABASES and FLUSH_TABLES, plus SELECT/LOCK TABLES on selected databases. The source receives no SUPER, CREATE USER or GRANT OPTION. Binlog replication access remains global; SELECT scopes and table filters are not a security boundary for the replication feed.

Target grants are SELECT, INSERT, UPDATE, DELETE and CREATE on selected databases. CREATE supports the current audit, checkpoint and JDBC schema-history preparation workflow. Optional partition maintenance adds ALTER and DROP; leave it off when this account does not need those operations. No global/replication/user-management grants are added.



Illustrative target setup; optional maintenance grants are explicitly selected.

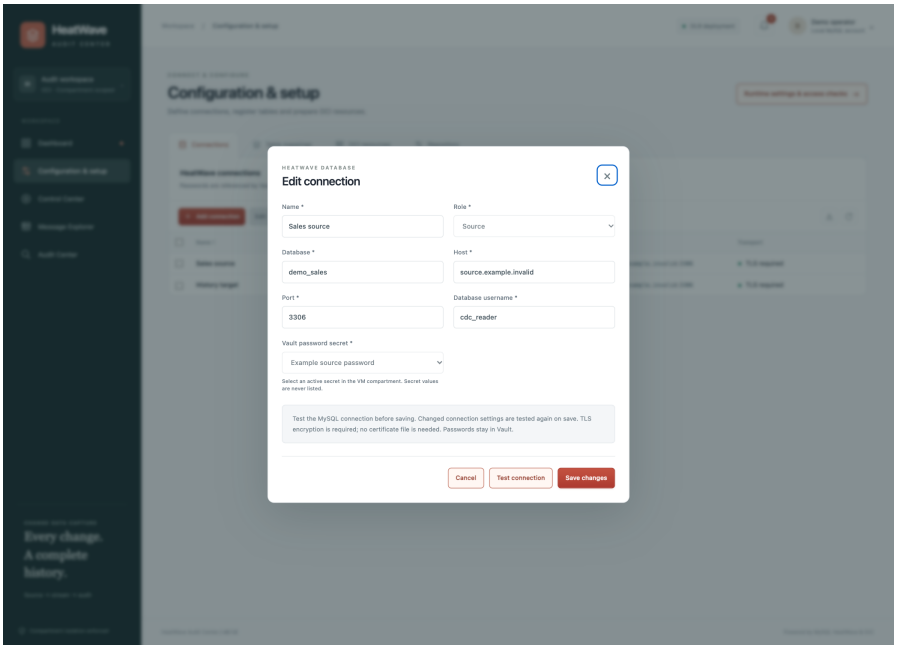
On success, the dedicated username and new Vault secret OCID replace the administrator inputs and the password field is cleared. Only the dedicated fields are saved and passed to workloads. Existing registered/deployed accounts are unchanged, and creation is not exposed through the restricted Edit dialog.

A partial failure can leave a secret or app-created account because MySQL and Vault do not form a distributed transaction. The safe error identifies the failed step and saved secret reference. Retry unchanged fields to resume the same setup. An existing unmanaged account is never adopted or reset. Non-secret progress is private under .data/account-setups/; it contains no administrator credential reference or password. See dedicated account setup and limitations for recovery after reload, database-pattern scope checks and permission details.

The same secret dropdown becomes **Administrator password secret** when dedicated-account creation is enabled. The newly created runtime secret is added and selected automatically after success. Empty/failed secret inventory is shown beside the field; secret values are never returned. See the compartment restriction and Appendix K for unsupported cross-compartment Vault/resource sharing.

USING THE APPLICATION

14. Edit registrations without breaking continuity



Prefilled edit dialog with fields locked after deployment history. Illustrative screenshot using the current templates and isolated sample data.

Select exactly one HeatWave connection, queue connection or table mapping and choose **Edit** in the table toolbar. More than one selection disables Edit. The app loads saved values into the corresponding dialog.

Registration condition	Allowed changes
Unused connection	Name, endpoint, role and credentials, subject to a fresh login test
Connection referenced only by undeployed drafts	Name and credentials; endpoint/role stay fixed
Queue connection referenced by drafts or app-managed topics	Name and supported credentials; selected service/resource stay fixed

Registration condition	Allowed changes
Undeployed mapping with no deployment history	Available source/target/topic choices and initial capture setting, after validation
Entry associated with a deployment attempt/history	Name only; stored capture path, offsets and row identity remain protected

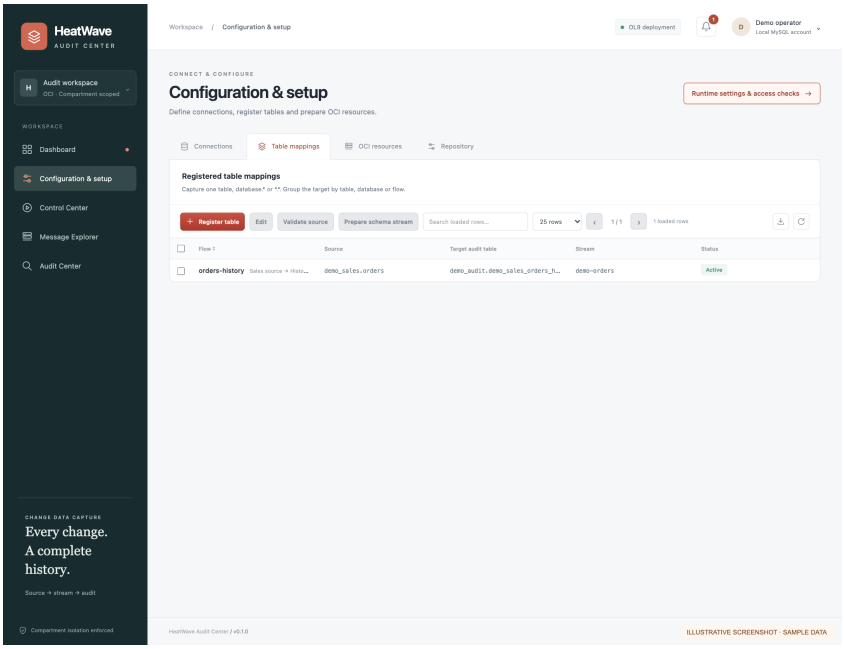
Stopping a deployed flow does not make its identity disposable. Use a new registration for a different source/target path; use **Capture start** for a supported explicit position change. Renaming preserves flow IDs, connector names, groups, history and fingerprints, and does not restart workloads.

Save revalidates permissions and state under server locks. A changed database connection is retested; live mapping/object validation is repeated. If another operator changed the entry after the dialog opened, refresh and reopen it. **Cancel** leaves the saved entry unchanged.

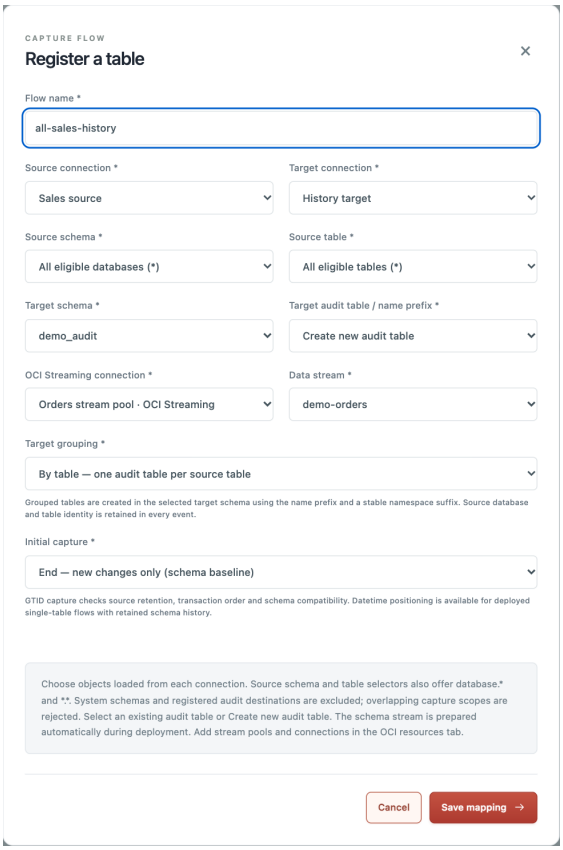
For password rotation on a referenced deployed connection, preserve the secret reference and coordinate rotation of its Vault value and affected workloads. Do not assume a name-only Edit can replace deployed credential references.

USING THE APPLICATION

15. Configuration: register a table mapping



Illustrative screenshot from current templates with isolated sample data.



Illustrative screenshot from current templates with isolated sample data.

Open **Table mappings** → **Register table** after source/target connections and the Streaming pool connection exist.

Field	Meaning
Flow name	Unique operational label
Source / target connection	Registered connections with the required role and Vault reference
Source database / table	One table, one database with `*`, or `*.*`; stable primary keys required
Target schema	Existing writable target schema
Target audit table	Compatible audit table or new root/prefix for generated destinations
Target grouping	By flow, by source database, or by source table
OCI Streaming connection	Active registered stream pool
Data stream	Existing prepared stream in the selected pool
Initial capture	End by default; Start snapshots current rows; Specific GTID starts after the validated processed set
GTID set and Check	Visible for Specific GTID; supports multiple UUID groups and intervals; shows executed/purged bounds

Save mapping creates a draft. Select the mapping for source checks and schema stream preparation. Deployment prepares missing schema infrastructure, not a missing selected data stream. Wait for the data stream to become active before deploying.

Overlapping source aliases, incompatible audit targets, reserved state tables and feedback paths are rejected. Deployed scope and grouping are locked to preserve history; register a new mapping for a different data path. See the source-scope and target-layout reference below.

CONFIGURATION AND CAPTURE DESIGN

16. Source scopes and target layouts

A mapping selects which source row changes belong to a flow and how the writer organizes their audit history. The target uses fixed audit-event JSON columns; it does not recreate each source table's columns.

Source selection

Source selection	Captured scope
`sales.orders`	One named base table
`sales.*`	Eligible base tables in one database
`*.*`	Eligible base tables across source databases

Use the database and table selectors in **Register table**. The wildcard is a whole-name choice; partial patterns such as `sales.order*` are not supported. Literal names are escaped when constructing Debezium's anchored regular-expression filters. Internal/system databases are excluded. Registered audit-target schemas on the same source server are excluded to prevent capture feedback.

Source checks inspect metadata and primary keys, not table contents. Current inventory is limited to 1,000 tables; a larger result is rejected rather than silently truncated. Every included table needs a complete stable primary key. A new table or database that matches the saved scope can be discovered while capture runs. A newly created keyless table is not silently ignored: unsupported row events stop the writer for review. Review filters and exclusions when adding new databases, audit destinations or broad source registrations.

Broad capture increases schema/change volume and may include sensitive application data. Choose the narrowest scope that meets the audit requirement. Source account permissions and metadata visibility must cover the intended databases; a wildcard does not grant MySQL privileges. Overlapping registrations on the same source and unsafe source-to-target feedback paths are rejected.

Target grouping

All three choices use the selected target connection and schema.

Grouping	Physical audit destination
By flow	One audit table for every source table in the flow
By source database	One generated audit table per actual source database
By source table	One generated audit table per actual source database/table pair

For example, `sales.orders`, `sales.customers` and `billing.orders` produce one, two or three physical audit tables respectively. By database does **not** create one target schema per source database. The registered target-table name acts as the root/prefix for generated destinations. Names include a deterministic hash of the complete source namespace and flow identity, remain within MySQL's name limit, and are recorded in durable route/ownership tables.

The writer creates a destination only after claiming ownership. It refuses an unowned existing physical table rather than treating it as an empty audit store. Interrupted creation can resume from its saved claim. The root table retains

the flow's retention anchor even when generated destinations hold the row events.

Each event retains its actual source database and table. Row-key hashes include that namespace, so identical primary keys in different tables remain distinct. The flow has one writer lease, one source offset namespace, and its prepared data and schema streams. Grouping does not create a connector or stream per table. Writes to several physical destinations are recoverable commits, not one atomic cross-table/source transaction. Replay checks prevent supported duplicate events when a later grouped write or checkpoint fails.

Browsing and operating boundaries

Audit Center combines the flow's owned destinations for history and charts. Filter by source database/table; a primary-key lookup must identify one actual table (and a database for `*.*`). Audit partitions lets you choose a generated physical table. Large JSON pages sort event coordinates before loading selected payloads.

End and initial Specific GTID are supported for wildcard mappings without copying existing rows. Start requests an initial row snapshot across the included scope and can be expensive. Normal restarts and image replacements retain full offsets. **Historical repositioning of an already deployed wildcard flow is disabled** until complete schema-baseline coverage across all included tables can be proven. The single-table historical preview does not imply that capability for a wildcard.

Automatic retention cleanup for grouped targets is disabled. Use a separately reviewed, coordinated maintenance procedure that preserves the flow's payload, deduplication, delivery, schema, routing and fragment evidence together. A row count or an old root table alone is not sufficient cleanup evidence.

Once deployment history exists, source scope and target grouping are locked. Register a new flow for a changed data path; do not edit identifiers under saved checkpoints. See capture position, Streaming and size limits and coverage limitations.

CONFIGURATION AND CAPTURE DESIGN

17. Streaming buffers, size limits and request control

The deployed path is source MySQL binlog → embedded Debezium Engine → native OCI Streaming → history writer → target audit store. The capture and writer use OCI SDKs over HTTPS with resource principals. No Object Storage overflow path is used. Ordinary events keep their original key and full Debezium envelope.

Why stage changes in a stream?

A durable stream lets capture publish while a writer is stopped, slow or being replaced. The writer later resumes from its target delivery checkpoint. Source capture and target apply can progress at different speeds, and short apply-side interruptions need not force an immediate source replay. Stream retention bounds this buffer: it is not an unlimited archive or a substitute for source binlogs.

There is a significant dependency in this implementation: the **source offset and Debezium JDBC schema history are stored in the target database too**. If the entire target database is unavailable, capture can also pause when it needs that state. Streaming decouples publishing from row apply, but does not make capture fully independent of the target database. Preserve enough source binlog retention for that outage and enough stream retention for the writer's recovery. An independent capture-state database would be an additional design change.

Which limits can change?

Boundary	Current behavior	Operator control
OCI message and PUT request	Service maximum remains 1 MiB; batching must fit the request too	Application configuration cannot raise the service maximum
Publisher request budget	At most 900,000 estimated encoded bytes, including keys and overhead	Fixed conservative application budget
Normal event	Original key/value in one message if it fits the publisher budget	No fragmentation for events that fit
Oversized event	Lossless framing, with chunks of at most 400,000 packed bytes	Automatic within the logical event limit
Logical event	Default 16 MiB for packed key, value and framing header	`streaming.max_event_bytes`: 1–64 MiB
Incomplete fragment custody	Default 128 MiB reserved per flow	`streaming.max_pending_fragment_bytes`: at least one maximum event, at most 1 GiB
Target database	JSON/document, packet, connection and storage limits still apply	DBA settings and supported service configuration; application limits do not override them

The publisher budget is deliberately below the service ceiling. An event smaller than 1 MiB in raw JSON may still need fragments because the key, UTF-8, base64 encoding, full envelope and request structure consume space. Inspect serialized bytes rather than character count. Oracle describes chunking and external payload storage as large-message designs; this application implements chunking. OCI publishing and large messages, PutMessages request limit.

Debezium envelope overhead

The `payload` contains the operation, before/after values and source coordinates. The outer `schema` describes the serialized fields and their types. A two-column update can therefore carry far more metadata than changed business values. Source metadata describes the event's origin and replication position; database passwords and connection secrets are not part of the row message.

This release retains the complete schema and both images for supported updates. It does not silently remove fields or change typed fingerprint semantics. Debezium JSON serialization can omit repeated schema descriptions, but a compact mode needs an explicit schema recovery/versioning contract and compatibility validation. Such a mode is **not implemented here**. Removing unchanged columns or the BEFORE image would also change what audit history can establish. Fragmentation preserves information; it is not compression and adds some bytes and processing work. Debezium Engine serialization.

Snapshots read current source rows directly. End and Specific GTID establish schema and follow binlog events without a row-copy snapshot. A broad wildcard scope still increases metadata and change volume; skipping the snapshot does not remove the cost of reading and serializing large row changes. Debezium MySQL connector.

Fragment delivery and recovery

1. The publisher measures the complete key/value. A normal event stays one message.
2. An oversized event is packed with the original key and split. Each part includes its index/count, declared size, message identity and part/full SHA-256 checksums. A stable routing key keeps the parts of that event in one

stream partition.

3. The source checkpoint advances only after every required part was accepted. A partially accepted request cannot acknowledge an unpublished source event.
4. The writer commits fragment custody before advancing the delivery checkpoint past an incomplete part. A restart can resume from those saved parts.
5. Only complete, validated events enter the normal audit writer. The original before/after data, schema and key are reconstructed; partial events never become audit rows. Completed payload blobs are released after durable delivery state proves completion. Compact receipts remain for replay checks.

A source restart can regenerate processing timestamps in an envelope. That can produce a different fragment generation for the same underlying source event. An older incomplete generation is retained even when a later complete generation arrives. Source-coordinate deduplication protects supported row events, but does not automatically reconcile every abandoned fragment set. Review retained custody and source/delivery evidence before explicit cleanup. The pending limit stops further custody instead of silently discarding incomplete data.

Complete receipts, the event deduplication ledger and audit payloads have different lifetimes. This release does not automatically prune every receipt. Grouped target retention also requires coordinated maintenance. Neither source transactions nor writes across multiple target tables form one distributed atomic transaction.

Fewer requests before retries

Several small events share one `PutMessages` request while remaining separate OCI messages. A request is flushed when its byte budget, record count or destination requires it. Data and schema streams require separate requests. No custom aggregate wrapper is added around normal events.

Runtime JSON setting under `streaming`	Default	Purpose
`put_max_records`	500	Maximum records per engine/publish batch, bounded by bytes
`put_requests_per_second`	4	Request ceiling per stream
`put_bytes_per_second`	850000	Smooth estimated encoded PUT bytes per stream
`get_max_records`	1000	Fetch more queued records in one GET
`get_idle_max_ms`	2000	Back off empty partition polls up to this delay
`get_bytes_per_second`	1500000	Pace reads by estimated response bytes per partition

Publishing uses the lower of the configured byte rate and 850,000 times the stream's partition count. A hot key can concentrate traffic in one partition, so increasing the rate or partition count does not guarantee higher usable throughput. Reading waits at least 250 ms between requests for the same partition and slows further for byte volume or idle queues. Backoff adds some latency when a quiet queue becomes active. Each source engine also has a bounded 64 MiB change queue.

Retries remain bounded fallback handling. HTTP success does not prove every PUT entry succeeded: the publisher checks individual results, including numeric `429`, and retries only rejected entries. A failed batch retains the source checkpoint. GET retries retain the same cursor; target checkpoints move only after durable custody or audit

commits. Partition throughput, API management quotas and workload lifecycle quotas are distinct constraints; identify the service and operation from the actual error rather than assuming PUT and GET share one global quota. OCI Streaming throttling.

The Control Center page polls every 15 seconds, but shared status samples limit OCI reads across tabs and web workers: up to 45 seconds for settled states, 10 seconds during transitions. A throttled status read is shared too; the page does not immediately duplicate it. **Checked** displays the actual sample time.

Save Runtime settings, then choose **Control Center → More actions → Apply stream settings**. This uses the selected workload images and replaces the writer before the source while retaining history and checkpoints. An interrupted operation can be resumed with the same action. Unchanged effective policy/image pairs are kept. The action permits unchanged or increased message/custody limits; reducing them requires a separately reviewed drain and retained-fragment reconciliation.

Publisher health counters include PUT requests, accepted physical messages, wire bytes, throttled entries and logical events. Writer logs report GET requests, records, bytes, empty responses, retries and throttles. Compare deltas for a known workload instead of relying on retry counts alone.

Target size and browse limits

The target stores keys, images and schema as JSON in a fixed audit structure. It is not a column-for-column replica of the source's wide row. InnoDB page/row rules therefore differ from the total JSON payload size, and large values can use out-of-page storage. This does not make JSON or network packets unlimited. A JSON value is constrained by `max_allowed_packet`; client/server packet settings, writer memory, serialization expansion and batch size must also be adequate. MySQL JSON limits, packet limits, storage requirements.

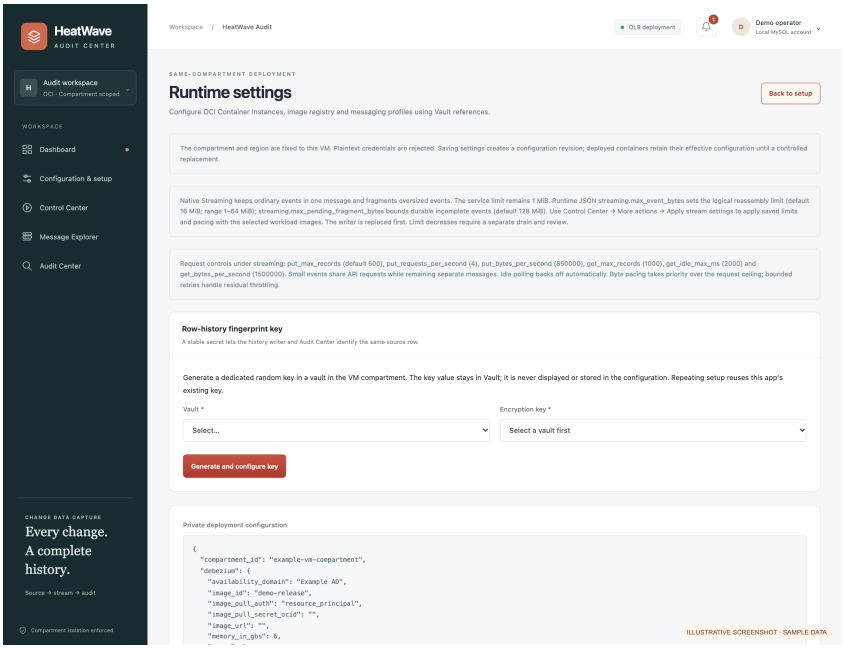
Raising the application event limit cannot raise the database's limits. Test the largest full update, including both images and schema, under representative load. The writer bounds batch bytes; one assembled event can exceed that ordinary batch budget while remaining subject to the logical event ceiling. Audit Center pages sort lightweight event coordinates before retrieving selected JSON payloads, avoiding a large JSON sort. Selected payloads are still loaded for the page: many very large events can produce a heavy browser response.

Validation scope

Live validation on 28 October 2026 delivered large Unicode CREATE, UPDATE and DELETE events exactly once in each of the three target layouts, verified partial writer reassembly recovery, and restarted source and writer without changing retained history. A separate burst of 100 row events plus one schema event used **2 PUT requests, 101 physical messages, zero throttled entries**. Normal events remained single messages. This is a functional batching result for that fixture, not a general throughput or latency benchmark.

USING THE APPLICATION

18. Runtime settings and fingerprint-key setup



Runtime settings with dedicated row-history key setup. Illustrative screenshot using the current templates and isolated sample data.

Open **Configuration & setup → Runtime settings & access checks**. Only administrators can save settings. The compartment and region remain fixed to the VM.

Field / action	Meaning and condition
Vault	Existing same-compartment vault for the fingerprint secret
Encryption key	Existing supported key in that vault; the app does not create an encryption key
Generate and configure key	Create/reuse this app's dedicated random key secret; requires Vault/key permissions
fingerprint_secret_ocid	Stable reference used by writers and row-history lookup; independent of database passwords
Private deployment configuration	Structured JSON of non-secret settings and Vault/resource references
Save configuration revision	Validate JSON/scope and save a recorded revision; never embeds plaintext credentials
Run read-only checks	Test service list/read access; success does not certify create/delete, container access or capture

Once configured, the app prevents removal or replacement of the fingerprint-secret reference. Preserve its value in Vault; key rotation/version routing for existing history is not implemented.

Settings that operators commonly review

`streaming` selects the native pool, logical/custody limits, PUT batching/pacing and GET polling policy. `debezium` and `processor` select workload network, shape, image and secret references. `processor.batch` and offset commit interval bound writer work. `mysql_tls` is global transport policy. `retention` contains explicit per-flow cleanup policies.

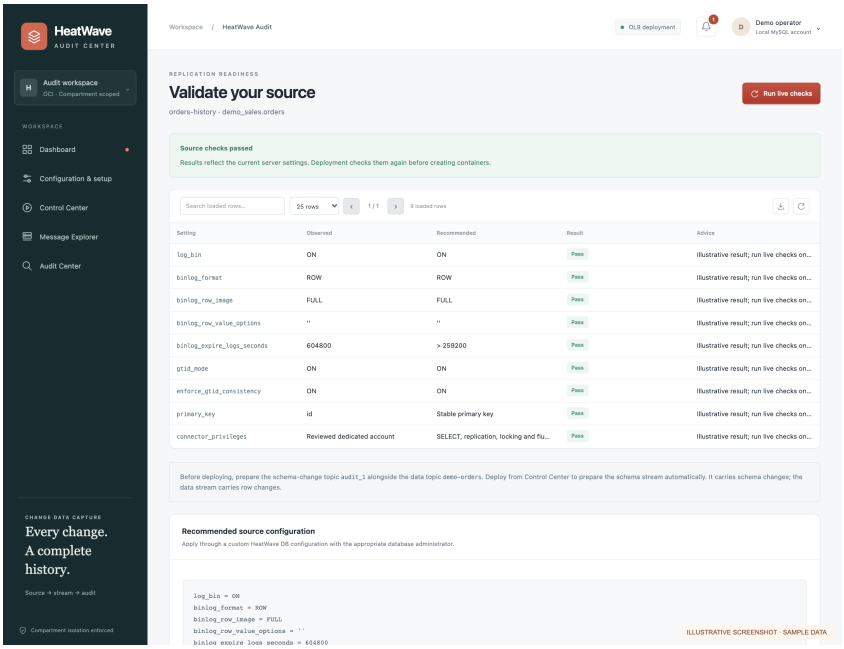
Saving a revision does not update effective settings inside existing containers. Use Control Center → More actions → Apply stream settings with compatible selected images. It replaces the writer first and retains checkpoints; limit reductions require a separately reviewed drain. Review continuity before changing queue incarnation, source identity, group or target; a configuration edit must not silently reset them.

The configuration contains sensitive deployment references even without passwords. Keep its real file on the VM and out of Git. Examples in this manual are placeholders.

Defaults: logical event 16 MiB, incomplete custody 128 MiB; PUT batch 500, request ceiling 4/sec and estimated bytes 850,000/sec per stream; GET batch 1,000, idle maximum 2 seconds and estimated bytes 1,500,000/sec per partition. See the complete Streaming reference below.

USING THE APPLICATION

19. Source validation and full row images



Live source validation results and recommended settings. Illustrative screenshot using the current templates and isolated sample data.

Open a selected mapping's **Validate source**, or Control Center → **More actions** → **Source checks** → **Run**. **Run live checks** reads the current source variables, table metadata and effective grants. It does not modify the source. Deployment runs readiness checks again.

Setting	Required / advised behavior
log_bin	ON
binlog_format	ROW
binlog_row_image	FULL
binlog_row_value_options	Empty; no PARTIAL_JSON
binlog_expire_logs_seconds	Greater than 259200 (three days); start sizing review at 604800 (seven days)

Setting	Required / advised behavior
gtid_mode / enforce_gtid_consistency	ON / ON
Primary key	Stable complete row key on each registered table
Connector privileges	Dedicated reviewed replication, read, flush and locking access

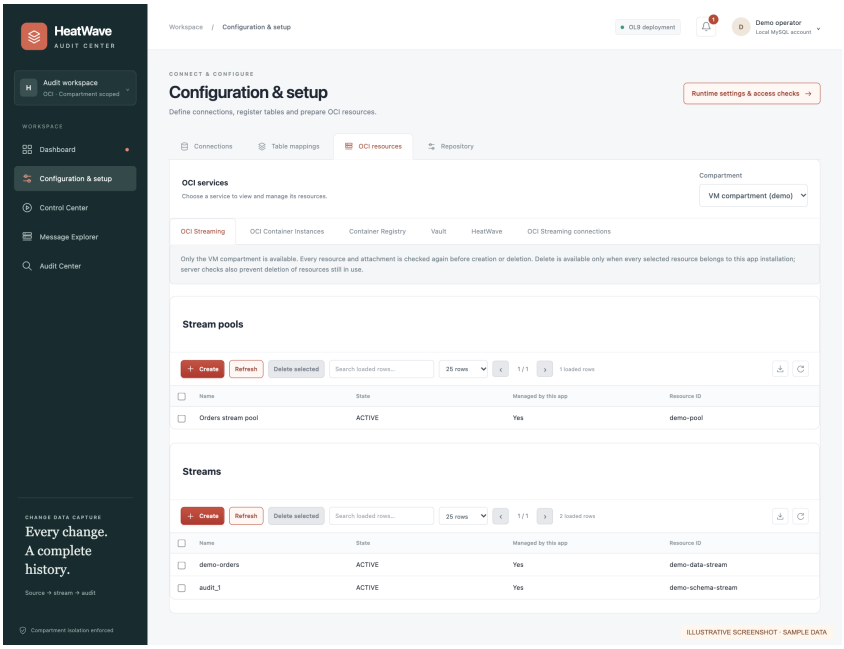
Observed reports the value read, **Recommended** states the requirement and **Advice** explains a correction. Any Action required result blocks deployment. A failed grant check can reflect role/scope ambiguity; have the DBA review it rather than broadening blindly.

Use supported HeatWave configuration parameters; some changes require a DB restart. Reconnect source application writers after changing session-sensitive row settings, then verify a controlled update. FULL can increase log volume significantly for wide rows/large values. Size source binlog storage, queue messages and target storage accordingly. No setting can reconstruct missing images in old logs.

Prepare the public schema topic before Deploy. It is distinct from the data topic and private recovery history.

USING THE APPLICATION

20. OCI resources: common controls



OCI resources with service tabs, compartment scope and inventory. Illustrative screenshot using the current templates and isolated sample data.

Open **Configuration & setup → OCI resources**. The service tabs are OCI Streaming, OCI Container Instances, Container Registry, Vault, HeatWave and Queue connections. Existing resources with lifecycle state DELETED are filtered from inventories.

Control / column	Meaning and condition
Compartment	Contains only the verified VM compartment; submitted changes cannot escape that scope
Create	Open the structured service form; requires administrator role, allowed scope and OCI create rights
Refresh	Reload the selected service inventory; submitted does not mean ACTIVE
Name / State / Resource ID	OCI display name, actual lifecycle state and resource identifier
Managed by this app	Whether this installation's provenance permits its lifecycle/deletion controls
Delete selected	Enabled only for eligible app-owned selections; backend also rejects in-use resources and rechecks scope/ownership
CSV icon	Download the loaded/selected inventory; not a resource backup

An app can select compatible existing resources without owning them. **Deletion stays disabled for resources not managed by this installation.** Creating with the same name does not transfer ownership. App ownership tags are not a substitute for IAM.

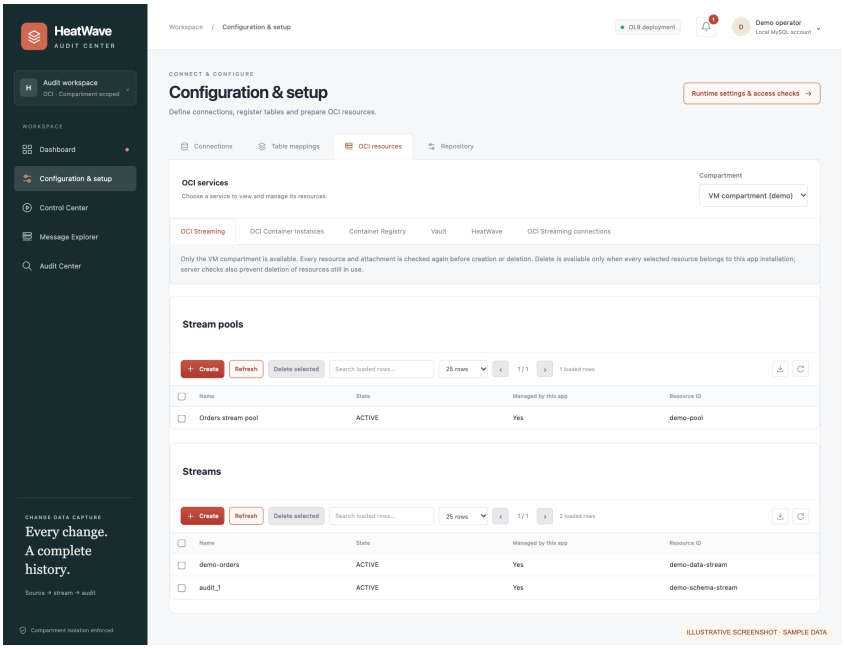
Resource creation is asynchronous and may incur charges. Review the notification/result, refresh and wait for the required lifecycle state. Ambiguous create results must be reconciled against inventory before retrying; do not repeatedly submit to force a duplicate. An authorization error shows the service issue rather than pretending the list

is empty.

The app provisions resources and attaches existing networking inside scope; it does not edit IAM policies or shared network rules. OCI service lifecycle actions are separate from removing local connection registrations.

USING THE APPLICATION

21. Native OCI Streaming: pools and streams



Illustrative screenshot from current templates with isolated sample data.

OCI Streaming uses its native HTTPS APIs. Select or create an active same-compartment stream pool and prepare the data stream plus the separate single-partition schema stream. Each flow binds delivery state to the actual stream IDs.

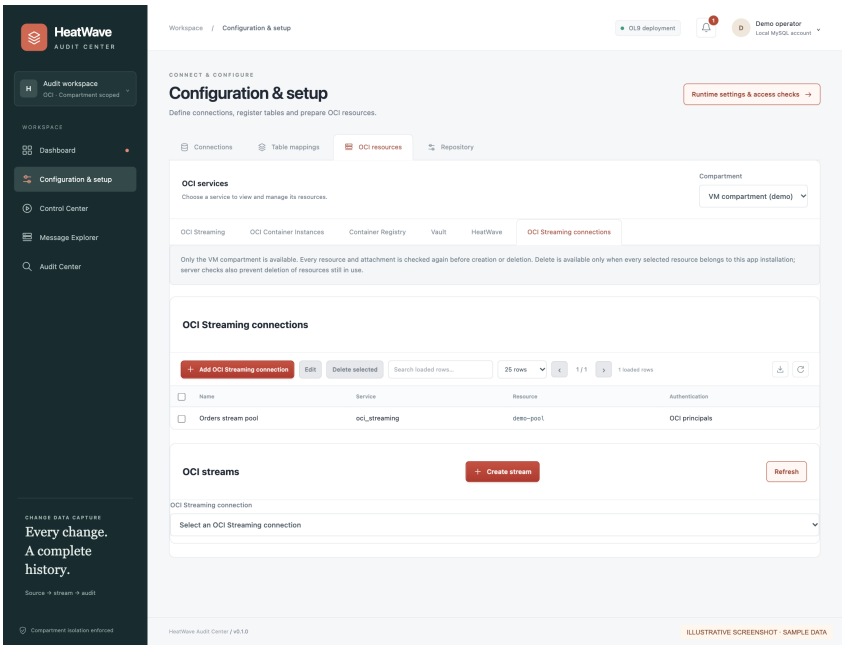
Resource / action	Meaning
Stream pool	Service grouping selected by the queue connection
Data stream	Row events; choose partitions and retention for measured load
Schema stream	`audit_<flow-id>`, exactly one partition
Refresh	Reload permitted service inventory
Create	Submit the supported typed resource form with reviewed scope
Delete	Guarded by compartment, ownership, lifecycle and in-use checks

The pool connection needs no Kafka/SASL credentials. The VM uses its instance principal; capture/writer use container resource principals. Source offsets and private schema history are in the target database. Public schema events remain in their own stream for the writer.

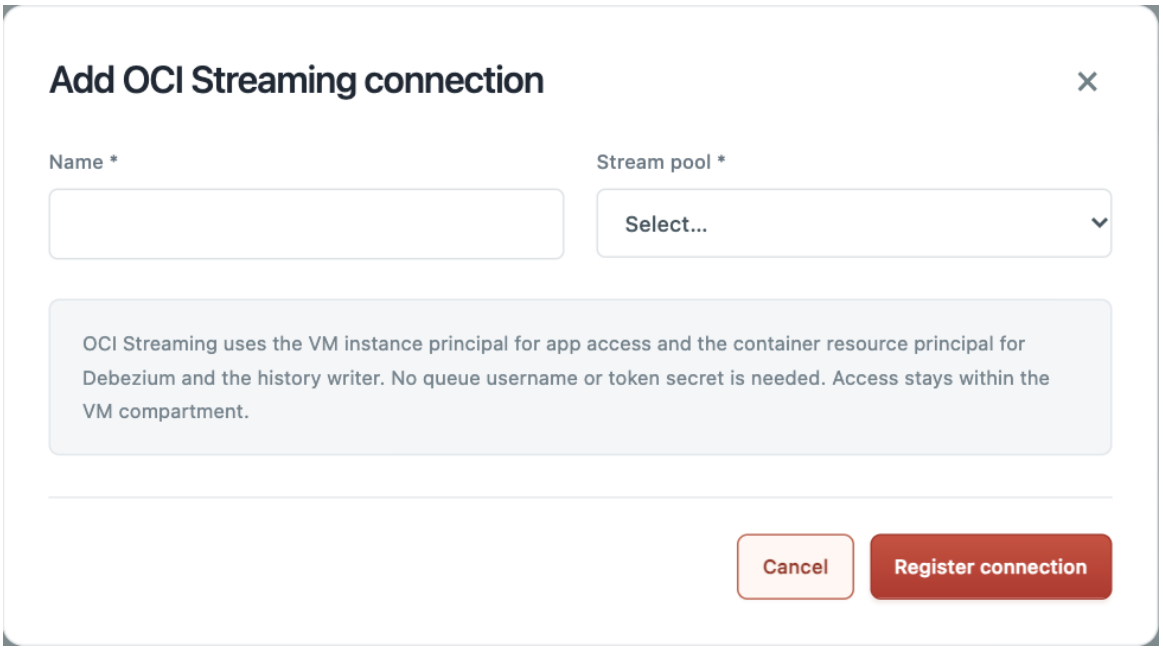
Retention is a bounded delivery buffer. Recreating a stream changes its identity and cannot silently reuse old checkpoints. A missing required data stream is a readiness error: prepare it in the selected pool. See the buffering and size limits reference for fragmentation and request pacing.

USING THE APPLICATION

22. Queue registrations and topic management



Illustrative screenshot from current templates with isolated sample data.



Illustrative screenshot from current templates with isolated sample data.

Open **OCI resources → Queue connections → Add OCI Streaming connection**.

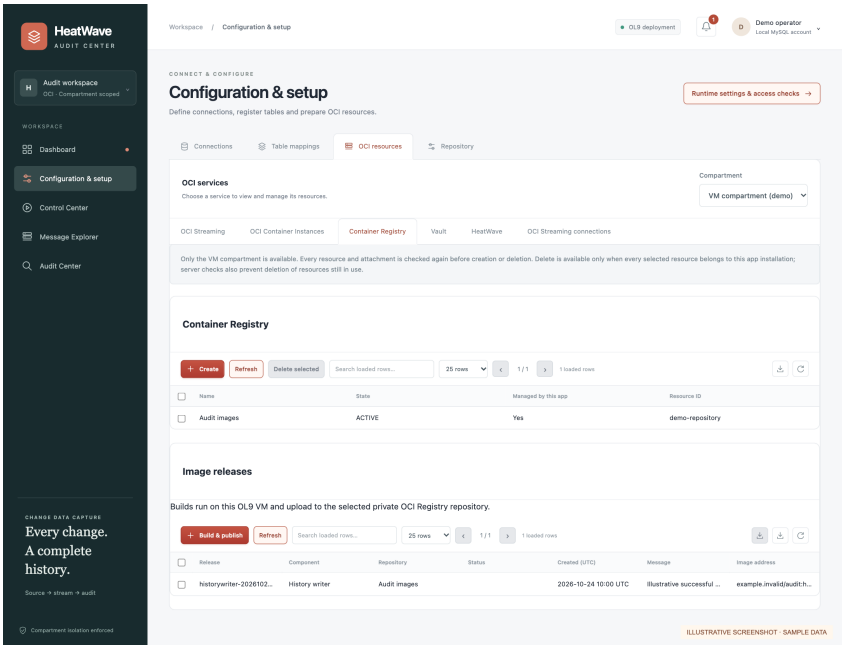
Field / action	Meaning
Name	Human-readable registration label
Stream pool	Existing active pool in the selected compartment
Register connection	Validate the pool and save the registration
Edit	Change permitted registration fields; in-use resource identity stays fixed
Delete registration	Remove an eligible local registration; does not delete the pool

No messaging username, password/token or external connector harness is requested. Source/target passwords remain separate Vault references.

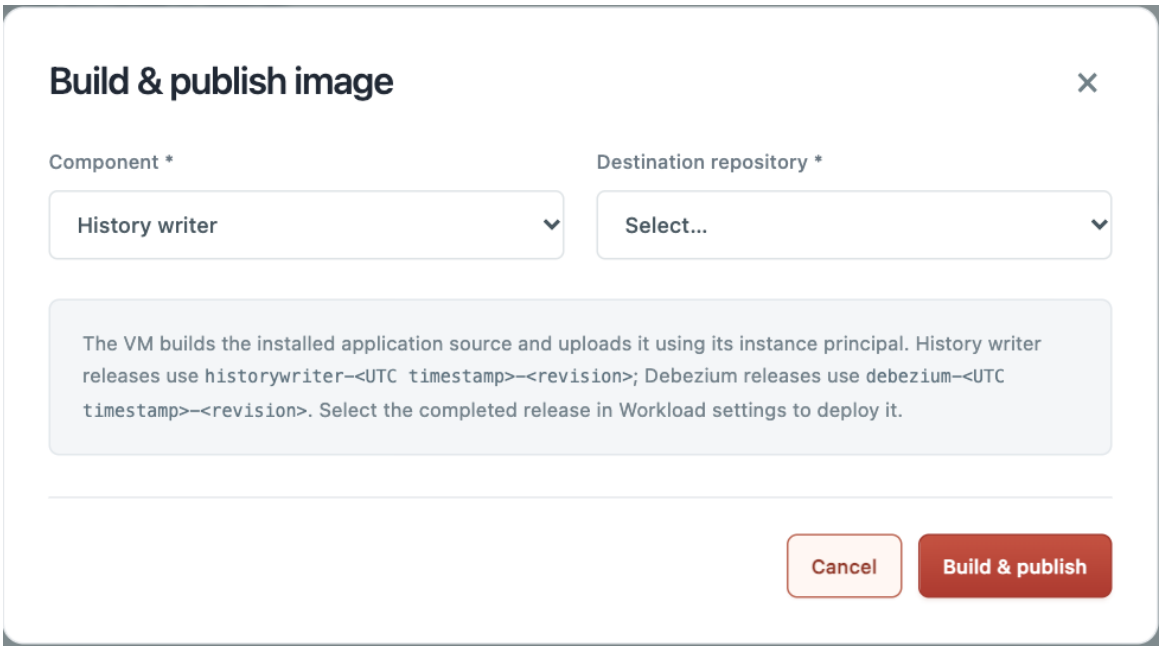
The streams panel loads streams for the selected connection. **Create topic** asks for name and partition count; actual OCI limits/capacity still apply. The action requests seven-day retention without changing an existing stream's policy. **Prepare schema topic** on a mapping prepares its one-partition `audit_<flow-id>` stream. Ownership/in-use checks guard deletion; deleting an unconsumed stream removes its retained delivery buffer.

USING THE APPLICATION

23. Container Registry and image builds



Container Registry and image release inventory. Illustrative screenshot using the current templates and isolated sample data.



Build and publish a component image. Illustrative screenshot using the current templates and isolated sample data.

In **OCI resources → Container Registry**, create/select a private repository. Repository creation asks for **Name**. The app uses the VM instance principal to build and upload images from the installed checkout on the OL9 VM.

Field / action	Meaning and condition
Component	History writer or Debezium; build both for a new deployment

Field / action	Meaning and condition
Destination repository	Same-compartment private OCI Registry repository
Build & publish	Submit to the separate image-builder service; requires it to be available
Refresh	Reload release/build states and result messages
Build log download	Select a build with a log; use the toolbar's log download control

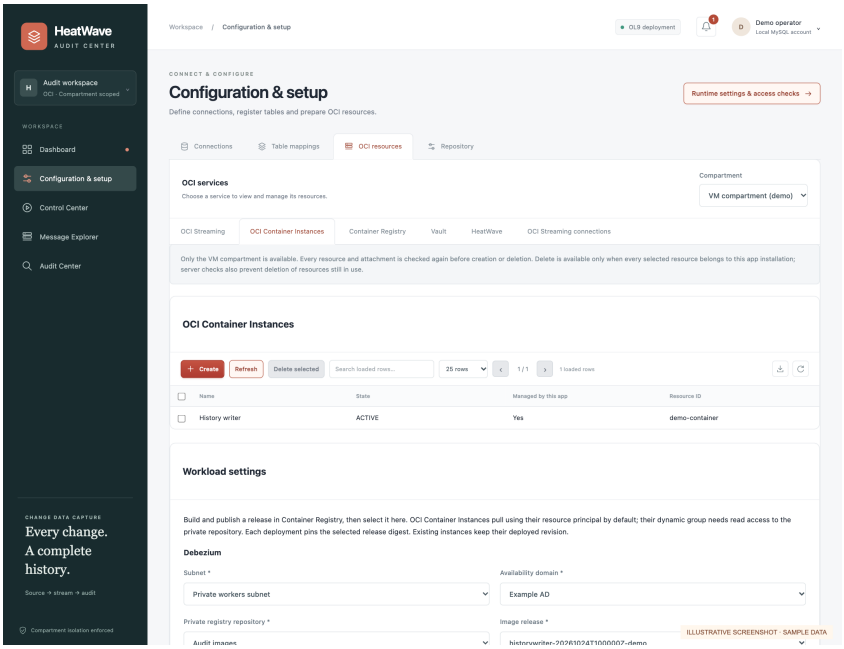
Releases are named `historywriter-<UTC timestamp>-<revision>` and `debezium-<UTC timestamp>-<revision>`. The image is uploaded during this build/publish operation. Workload settings select the completed release; deployment pins its registry digest internally. You do not type a new digest to initiate a build.

A failed build is not deployable. Review its safe log, dependencies, registry permissions and runtime configuration. A missing `AUDIT_CONFIG_FILE` means the builder/runtime environment needs correction; resubmitting without it will repeat the failure. Do not rerun setup while a build is active.

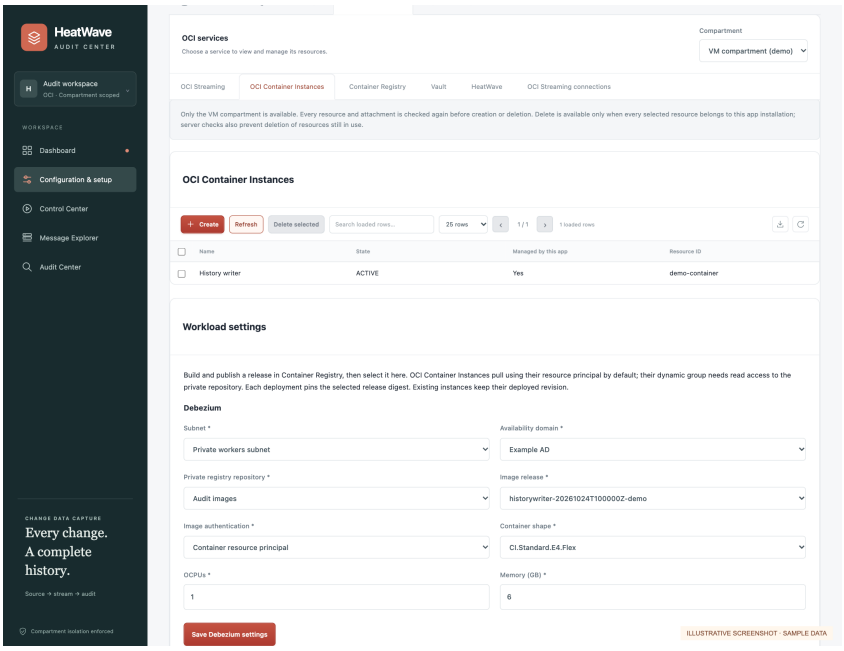
A newly completed build does not replace running containers. In Control Center select one flow, choose the appropriate **Update image** action, then choose the exact published release in its dialog. Workload settings retain initial-deployment defaults. Resource-principal image pulls need repository permissions for the container principal, independently of successful VM image publication. A static pull secret is optional only when that alternate mode is explicitly selected.

USING THE APPLICATION

24. Container Instances and workload settings



OCI Container Instances inventory. Illustrative screenshot using the current templates and isolated sample data.



Workload settings for separate Debezium and history-writer roles. Illustrative screenshot using the current templates and isolated sample data.

The **OCI Container Instances** tab lists instances and provides a creation form containing **Registered flow**. Creating through that form deploys the selected flow after readiness checks; it is not a generic arbitrary-container editor.

Configure both role forms in **Workload settings**:

Field	Meaning / condition
Subnet	Existing private worker subnet in the VM compartment
Availability domain	Available placement for the instance
Private registry repository	Permitted repository containing the component's image
Image release	Completed appropriate component release; digest pinned on deployment
Image authentication	Container resource principal by default, or explicit Vault pull secret mode
Image pull secret	Required only for Vault mode; hidden/disabled for principal mode
Container shape	Available Container Instances shape
OCPUs / Memory (GB)	Requested workload capacity within form/service bounds

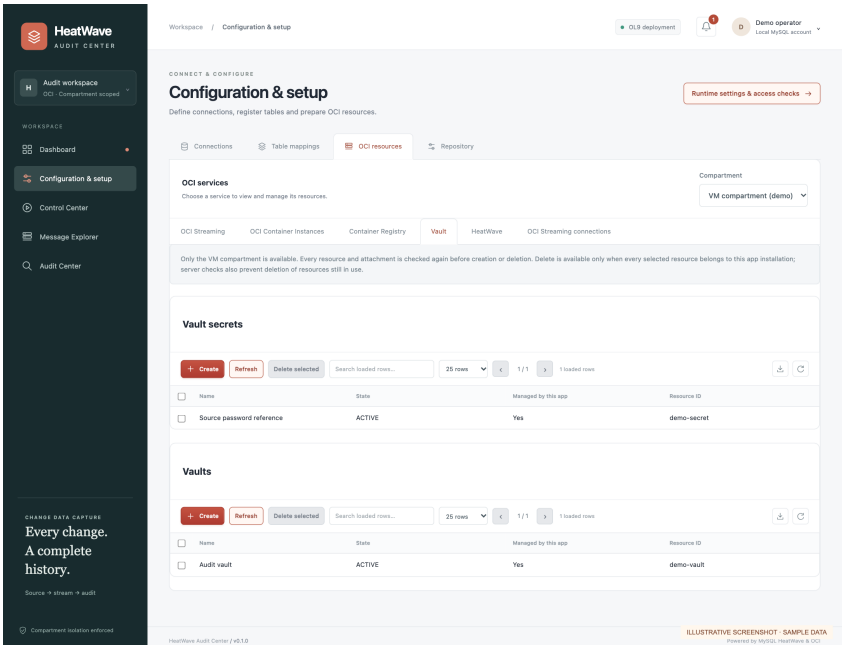
Save Debezium settings / Save History writer settings records the selection and configuration. Existing containers keep their deployed image and effective revision. **Start** resumes their existing deployment; **Update image** opens a per-flow release picker and performs the controlled replacement. Choosing a release there does not change these defaults or other flows.

Give Debezium sufficient memory for connector/schema state and expected event sizes. Give the writer sufficient memory for bounded batches and target connections. Shape changes do not create automatic scaling; review replacement behavior and capacity with an acceptance test. The current lifecycle model tracks one connector and one writer per flow.

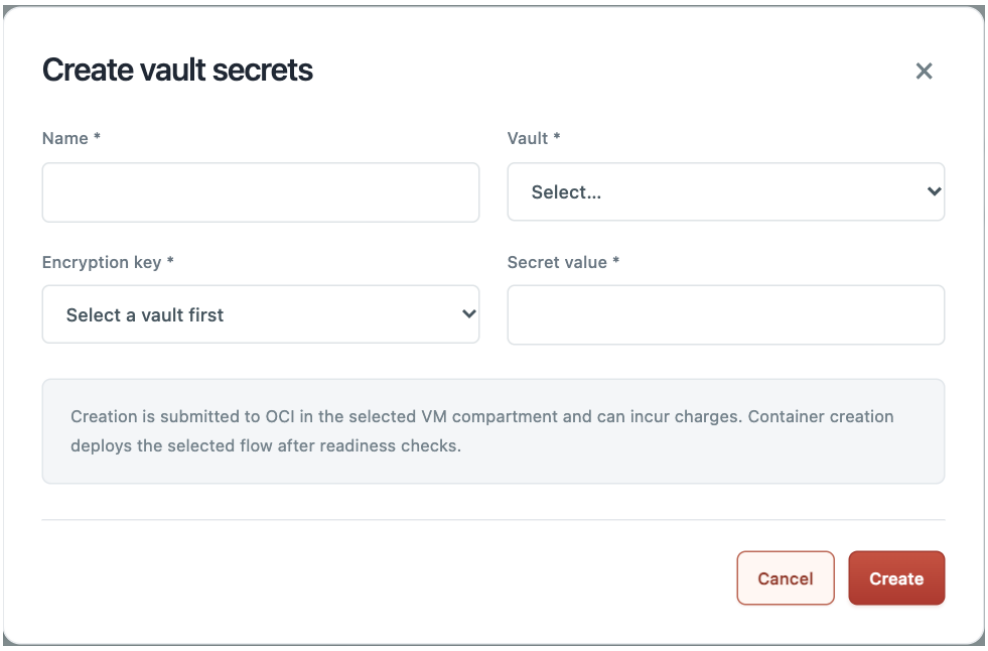
Use flow controls for app-owned workloads. Resource inventory deletion and retired-container termination are guarded and should not be used as an offset-reset mechanism.

USING THE APPLICATION

25. Vault and password references



Vault and secret inventories. Illustrative screenshot using the current templates and isolated sample data.



Create a secret using an existing vault and encryption key. Illustrative screenshot using the current templates and isolated sample data.

Use the **Vault** tab to list vaults and secrets. New vault creation asks for **Name**. Secret creation asks for:

Field	Purpose
Name	Display name describing the secret purpose
Vault	Existing permitted vault in the VM compartment

Field	Purpose
Encryption key	Existing supported key belonging to that vault
Secret value	Enter privately; used to create the secret, not stored as a connection password

Create submits the chosen resource; **Cancel / ×** discards the form. **Refresh** checks readiness. Selecting the resource later uses its identifier, not its plaintext value. Secret creation needs Vault/key encryption rights; reading existing secret metadata and retrieving secret bundles are different permissions.

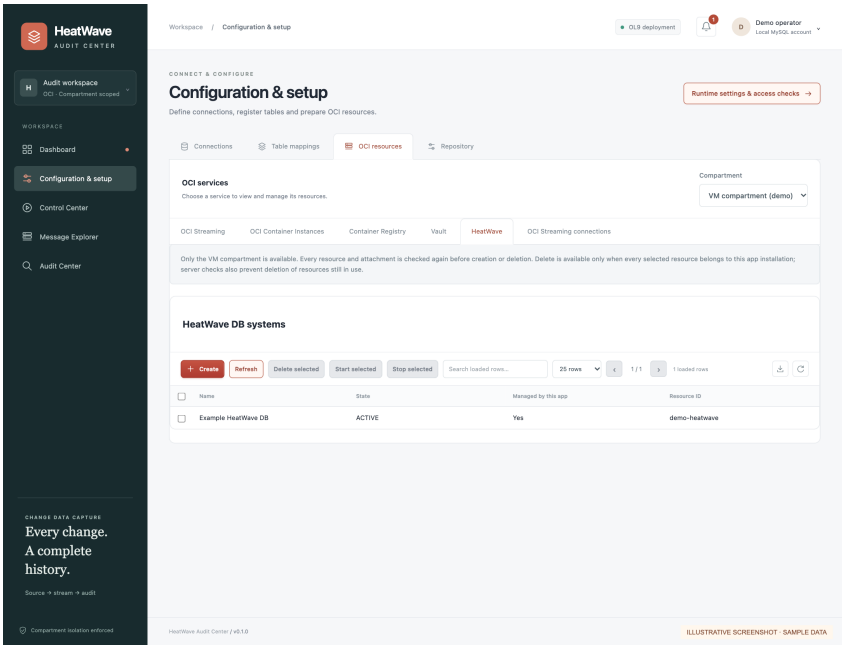
Use separate purpose-specific secrets: source DB password, target DB password and the dedicated row-history fingerprint key. An optional registry pull secret is used only for its configured authentication mode. The key-generation screen creates the fingerprint secret; it should not contain a database password.

Delete controls are app-ownership/in-use guarded. Vault/secret deletion is scheduled according to the OCI operation; it is not always immediate. Do not schedule deletion of the fingerprint secret or a credential needed by retained history/recovery. Review actual OCI state after submission.

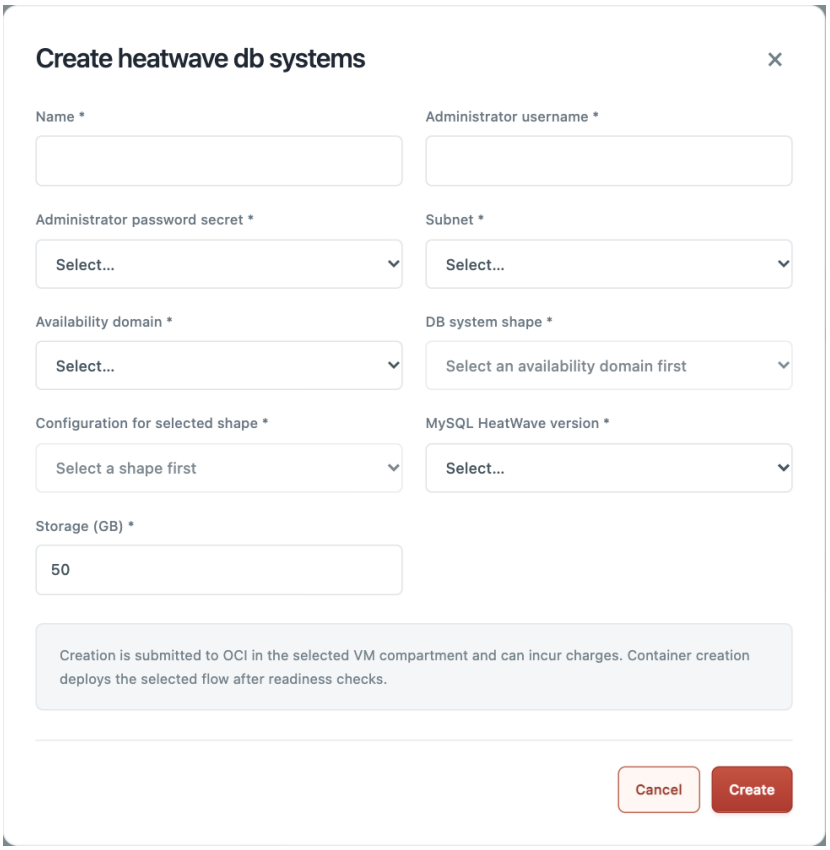
Workloads resolve permitted secrets through their own resource principals. Successful VM Test connection does not prove that a container can read the same secret. Secret rotation and effective workload refresh need an explicit controlled procedure.

USING THE APPLICATION

26. HeatWave DB resource controls



HeatWave DB system inventory. Illustrative screenshot using the current templates and isolated sample data.



HeatWave creation with dependent shape, configuration and version dropdowns. Illustrative screenshot using the current templates and isolated sample data.

The **HeatWave** resource tab manages DB systems independently of the Connections registrations. A connection can use an existing supplied endpoint without entering its DB system OCID.

Create field	Meaning / condition
Name	DB system display name
Administrator username	Initial DB administrator account
Administrator password secret	Vault reference for the account password
Subnet / Availability domain	Existing permitted attachment and placement
DB system shape	Loaded available shapes after placement selection
Configuration for selected shape	Compatible shape-matched default/custom configuration
MySQL HeatWave version	Loaded available service versions
Storage (GB)	Requested supported storage size; form minimum 50 GB

Create submits the DB system and may incur charges; it does not automatically register a table flow. Wait for readiness, confirm endpoint/schema/accounts, then add the appropriate connection. An Oracle-owned default configuration is a service-catalogue choice; selected custom resources and attachments must satisfy the app's scope checks.

Select 1–25 distinct same-compartment DB systems. **Start selected** requires every selected DB system to be INACTIVE; **Stop selected** requires every selected system to be ACTIVE. These actions do not require app ownership, whereas deletion does. Stopping interrupts database connections and capture/writes. Operations can be asynchronous and conditional. HTTP 412 can reflect a lifecycle/precondition conflict; refresh state and review the request details instead of repeating blindly.

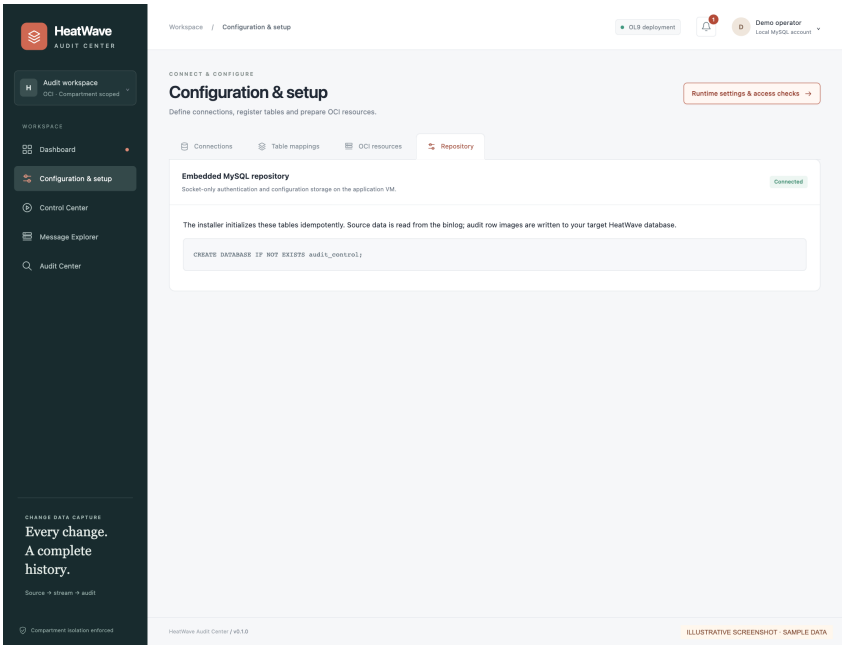
Delete selected is disabled for resources not managed by this installation and rejected when referenced/in use. Deleting a DB system is separate from deleting a connection or flow registration and can destroy data. Preserve needed audit evidence and recovery state before any resource cleanup.

HeatWave operations omit ETag / If-Match

The app does not send ETag/If-Match for HeatWave OCI operations. Fresh compartment/state checks, deletion ownership/dependency checks and request IDs remain. Start/Stop uses an idempotency token. HTTP 412 was observed even after refreshing GET responses; compression could alter a representation ETag, and resource-version mismatches also occurred without it. The app does not modify an opaque ETag or rely on a refresh to solve every failure. See Appendix J.

USING THE APPLICATION

27. Embedded repository page



Embedded MySQL repository information. Illustrative screenshot using the current templates and isolated sample data.

Open **Configuration & setup → Repository** to view the local control-repository information and initialization SQL. The installer initializes the repository tables idempotently.

The local socket-only MySQL stores application accounts, connection references, table mappings, configuration revisions, operation records, resource provenance and build registrations. It is not the source binlog database and not the target audit payload store.

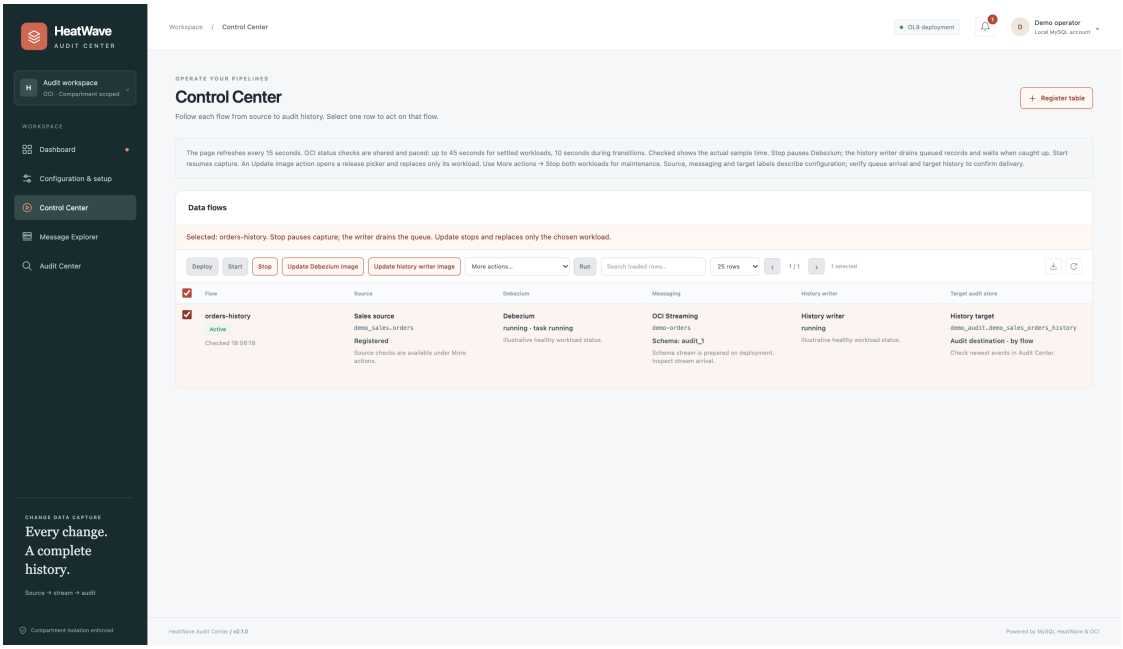
The **Connected** badge describes the local repository context; it does not prove source or target connectivity. Source data is read through Debezium from the registered source; row images are written to the selected target HeatWave schema.

There is no arbitrary SQL execution or full database-backup action on this information page. Use approved database backup tooling for consistent local-repository backups, together with the private runtime configuration and app installation identity. Restoring only the application source without its private identities/state does not establish capture continuity.

Do not copy local repository files, `.runtime.env`, sessions, TLS keys or an app-instance identity into a new independent deployment. Refer to the backup/recovery appendix when recovering the same installation.

USING THE APPLICATION

28. Control Center: flow row and status



Control Center with one selected flow and component status blocks. Illustrative screenshot using the current templates and isolated sample data.

One row shows Flow, Source, Debezium, Messaging, History writer and Target audit store. Tick exactly one flow checkbox to enable its eligible actions. Multi-selection can help table exports but does not enable multi-flow lifecycle actions.

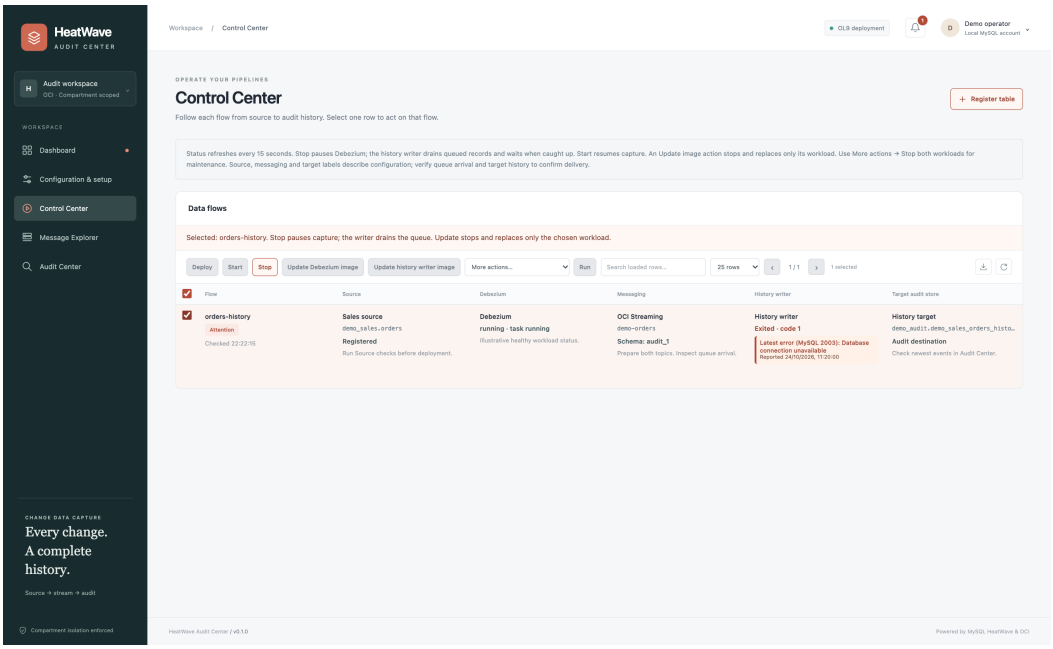
The Debezium block reports container/process and connector-task state, whether the deployed image differs from Workload settings defaults and a next-step tip. The writer block reports process state and a tip. Source, messaging and target labels describe configured endpoints; they are not delivery measurements. Status checks refresh approximately every 15 seconds while visible.

State	Meaning / next action
Draft	Saved mapping; validate source, messaging, target and images before Deploy
Deploying / starting / stopping	OCI/process transition; refresh and inspect if persistent
Active	Fresh running process/task reports; still verify queue and target evidence
Pending	Incomplete deployment or unavailable task evidence; inspect missing workload/log access
Attention	Failed/exited process, stopped task or unexpected inactive workload; follow its tip
Capture paused	Debezium intentionally stopped; writer drains the queue and waits
Stopped	Both registered workloads inactive; Start resumes deployed images/checkpoints
Unavailable	Status cannot be established, including access/session errors; do not assume healthy
Retired	Capture retired; audit history preserved; termination/deletion are separate actions

An inactive writer cannot consume. To select a fixed release, use Update history writer image rather than repeatedly starting an old failing image. A running connector task plus a running writer still requires target checkpoint and newest-event checks.

Latest error belongs to the process

Each process block shows a compact latest-error summary and report time alongside state/exit code. Click the process field to open its full value, including the failed step and recovery guidance. For example, target-connect MySQL 2003 means the configured database is unreachable; verify DB availability before resuming the writer. Current images emit structured diagnostics. Older images may have logged only a database exception class; the app does not invent a missing numeric code. Log access can also be unavailable. A timestamped latest error can be historical after recovery.



Illustrative failed-writer example: MySQL code and report time; click the field for the failed step and next action. No live credentials or row images are shown.

The page polls every 15 seconds. Shared OCI samples can be 45 seconds old for settled workloads or 10 seconds during transitions. Checked shows the actual cloud sample time. An image difference from the saved default is informational, not a source-database failure.

USING THE APPLICATION

29. Control Center: action conditions

All lifecycle actions require an administrator, exactly one selected flow and fresh supported state. The backend repeats readiness, scope, ownership and flow-lock checks.

Action	Available condition and effect
Deploy	Non-retired flow missing one/both registered workloads; checks readiness and creates missing components

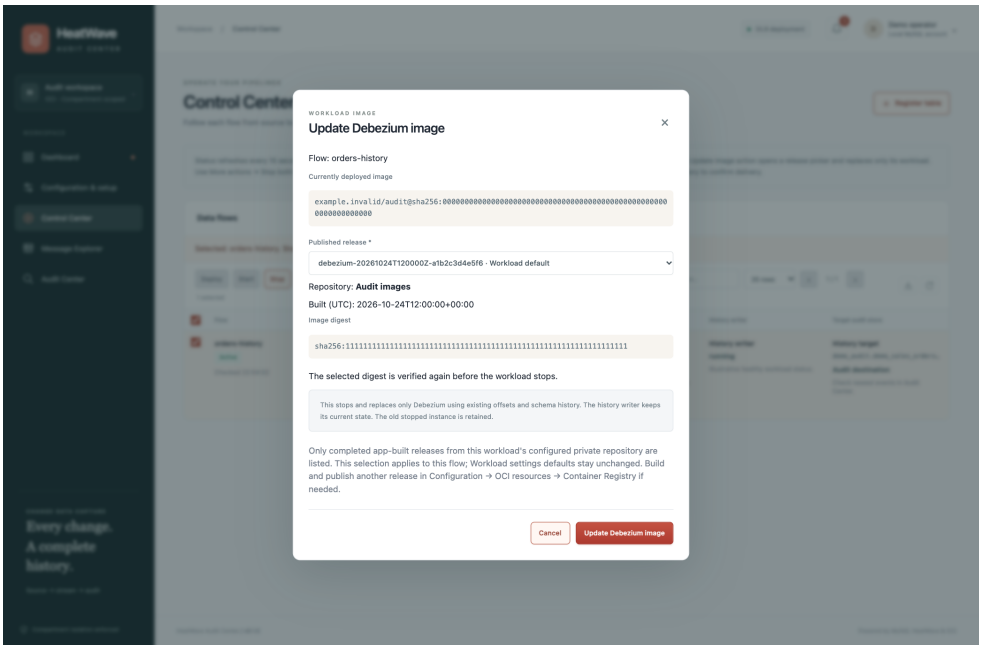
Action	Available condition and effect
Start / Resume	Both components registered, supported ACTIVE/INACTIVE states, at least one inactive; resumes saved images/checkpoints
Stop	Debezium instance ACTIVE; pauses only capture while writer continues draining and waits
Stop both workloads	At least one workload ACTIVE; explicit maintenance stop of both components
Update Debezium image	Deployed Debezium ACTIVE/INACTIVE; open the release picker, select a different published release, then preflight, stop, wait and replace Debezium only
Update history writer image	Deployed writer ACTIVE/INACTIVE; open the release picker and select a different published release; replace writer only; capture may keep adding messages
Apply stream settings	Both workloads in ACTIVE/INACTIVE states; apply saved message/pacing policy and selected images, writer first; same image digest is allowed; reductions require separate drain/review
Restart	Both components registered with supported ACTIVE/INACTIVE states; restart with existing state, no new position
Source checks / Capture start / Audit partitions	Open the selected flow's corresponding page; that page independently validates actions
Prepare schema topic	Non-retired flow with a queue registration; prepare required schema stream/topic
Refresh status	Request current health; does not reset offsets
Retire	Non-retired flow with registered workloads; stops capture/apply and preserves evidence
Terminate retired containers	Retired flow still has registered containers; terminate guarded resources
Delete mapping	Retired flow with no registered containers; remove the registration, retaining target evidence

For More actions, select the action and click **Run**. A disabled Run means no eligible single flow/action is selected. Some actions require a confirmation describing their effect. The Update buttons open a release picker for running or stopped workloads. Choose a different available completed component release and submit; the action performs the required stop itself.

If shutdown exceeds the bounded wait, allow OCI to reach INACTIVE, reopen Update image and choose the intended release again. The app refuses to create a duplicate active replacement before confirming shutdown. A failed replacement can leave the old instance stopped; inspect the message and retry after correction.

Stopping capture while the source keeps changing consumes the source binlog-retention window. Stopping the writer consumes queue-retention capacity. Plan either pause against measured lag and retention, not container status alone.

Choose the exact image for this flow



Illustrative image-update dialog showing the deployed digest and the exact selected release.

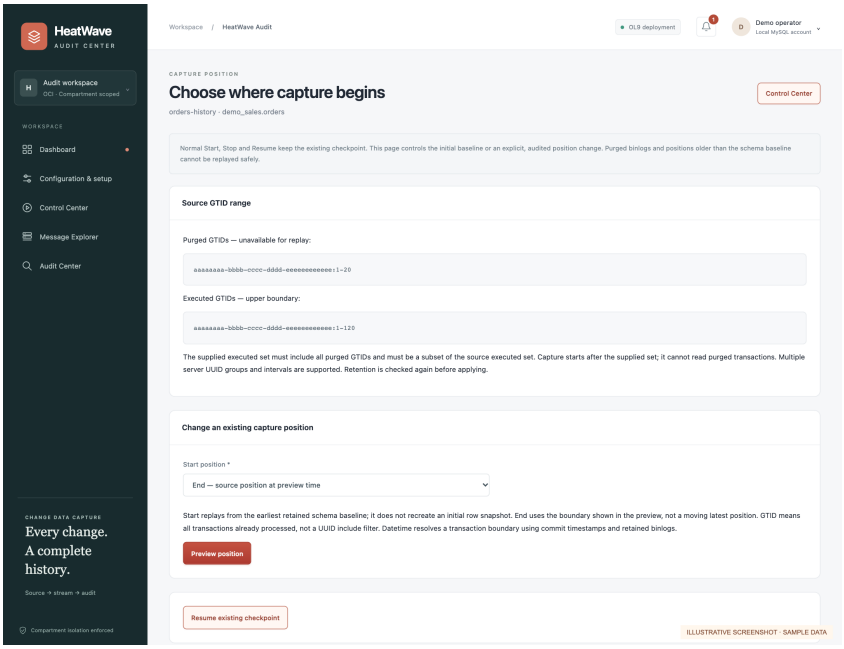
Field / action	Meaning and condition
Currently deployed image	Digest-pinned image read from the registered Container Instance
Published release	Required dropdown of completed app-built releases for this component in its configured private repository and VM compartment/region; deleted, missing and current images are excluded
Repository / Built (UTC) / Image digest	Details of the chosen release; a Workload default label identifies the saved initial-deployment default
Update image	Enabled after choosing a different release; validates live availability/digest and supported state before stopping the selected workload
Cancel / ×	Close without submitting an update

The release ID and the instance shown in the dialog are checked again under the flow lock. A stale dialog is rejected if another update has replaced that instance. The selection is local to this flow and does not overwrite Workload settings defaults. An earlier published release can be chosen after compatibility review; rolling back an image does not reverse schema changes or source transactions.

Workload default differs means the deployed digest differs from the saved default; it is not a claim that the default is newer or that an update is mandatory. The old stopped instance remains for investigation or explicit removal after replacement health and checkpoint progress are verified.

USING THE APPLICATION

30. Capture start, GTID sets and purged history



Illustrative screenshot from current templates with isolated sample data.

A new mapping supports End (default), Start, and Specific GTID. End and GTID establish schema without scanning current rows. Start reads current source rows directly and then follows the binlog. Large snapshots require source capacity, locking and retention planning. The target's fixed JSON audit structure does not require a row-copy baseline to record later changes.

For a deployed **single-table** flow, Capture start can preview supported retained Start/End/GTID or UTC datetime boundaries with proven schema coverage. Historical repositioning is disabled for deployed wildcard flows. Normal Start/Resume/Restart preserves full offsets and does not repeat initialization.

A GTID set means transactions **already processed**. For example:

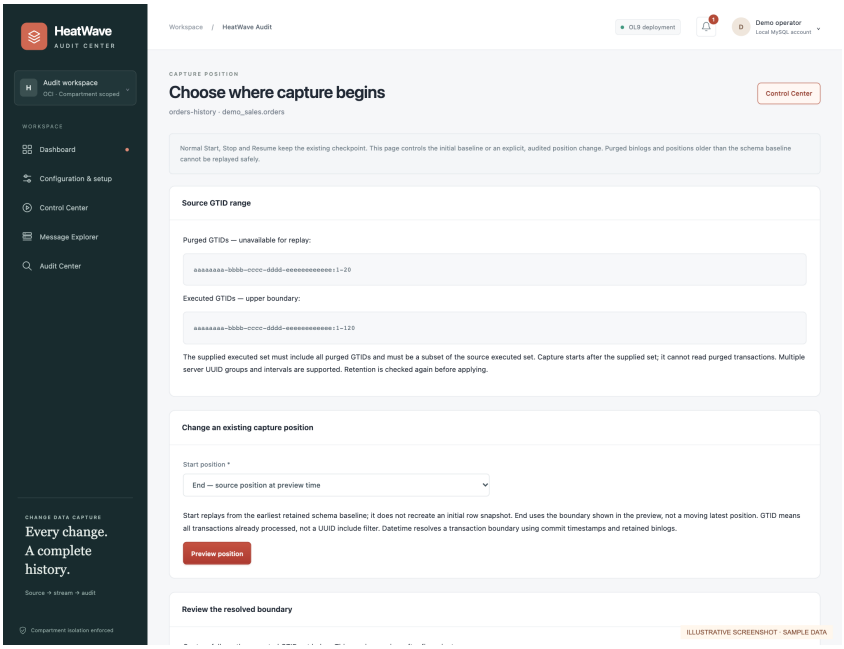
```
11111111-1111-1111-1111-111111111111:1-32:40-45,
22222222-2222-2222-2222-222222222222:1-18
```

Each UUID can have multiple intervals. Inspect executed, purged and selected sets. Required containment is `purged ⊆ selected ⊆ executed`: no transaction still needed for capture may have been purged. Already-processed purged transactions belong in the selected set. A complete supported transaction-order prefix and compatible schema boundary are also required; interval membership alone is insufficient. Unsupported holes and unavailable schema/binlog evidence are rejected.

A row event's `source.gtid` identifies its transaction, not the whole processed set or a unique row event. Keep file/position/event/row coordinates in the durable source offset too. See capture position and recovery.

USING THE APPLICATION

31. Apply a reviewed capture position



Illustrative screenshot from current templates with isolated sample data.

For an eligible deployed single-table flow, Preview position resolves and checks the proposed boundary without moving capture. Review source identity, full offset, retained schema evidence and the operational reason. Preview is actor/flow bound, expires after five minutes and must be regenerated if stale.

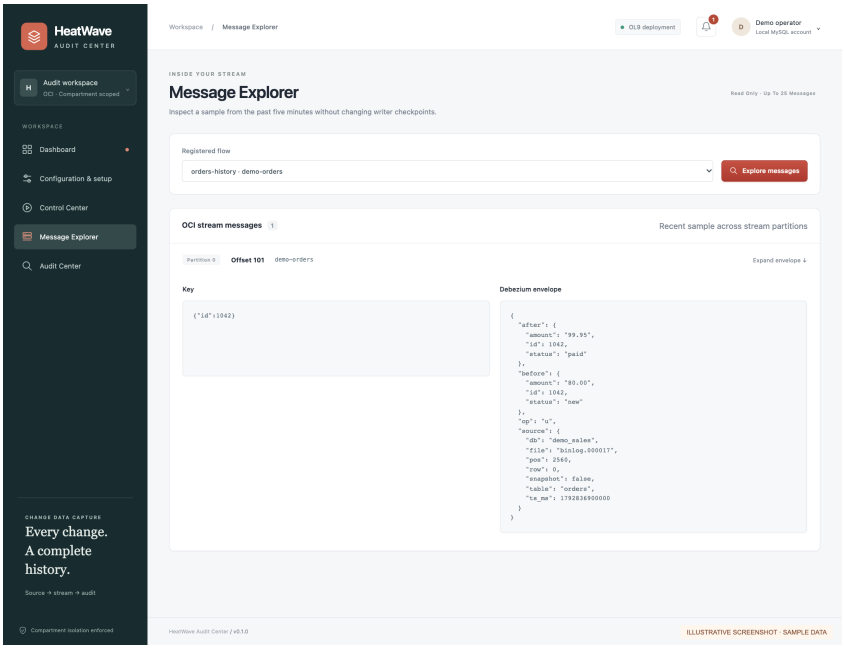
1. Generate a fresh preview on the intended flow.
2. Review replay or skipped-history effects and tick the acknowledgement.
3. Choose **Apply this position once**. Source/schema checks run again.
4. Inspect the latest request result, source task, stream and target progress.

The runtime controls the embedded engine through its loopback native capture API, saves/read-checks full offsets and resumes from the reviewed position. The OCI instance remains running. Applied requests are not repeated on ordinary restart. Writer checkpoints and existing target history are retained.

An ambiguous failure can leave capture stopped for review. **Resume existing checkpoint** resumes the stored offset; it does not promise the former position. A retired flow cannot reposition. Wildcard historical repositioning remains disabled. Do not reset position to fix an unrelated writer or permission failure.

USING THE APPLICATION

32. Message Explorer



Read-only sample with an expanded Debezium envelope. Illustrative screenshot using the current templates and isolated sample data.

Choose **Registered flow**, then **Explore messages**. The page reads a bounded recent sample of up to 25 messages across the data topic's partitions. It does not advance or commit the history writer's consumer offsets.

Display	What it establishes
Partition / Offset / Topic	Queue delivery coordinates for this sample
Key	Complete row key serialized by the producer; null is meaningful for some non-row records
Debezium envelope	Operation, before/after payload and source coordinates; expand the message row to inspect it

Use the page to establish whether the data topic contains a recent event for a controlled source change. An empty sample does not prove the entire queue is empty or that no earlier retained events exist. Sampling is not a full message search, lag dashboard or replay action.

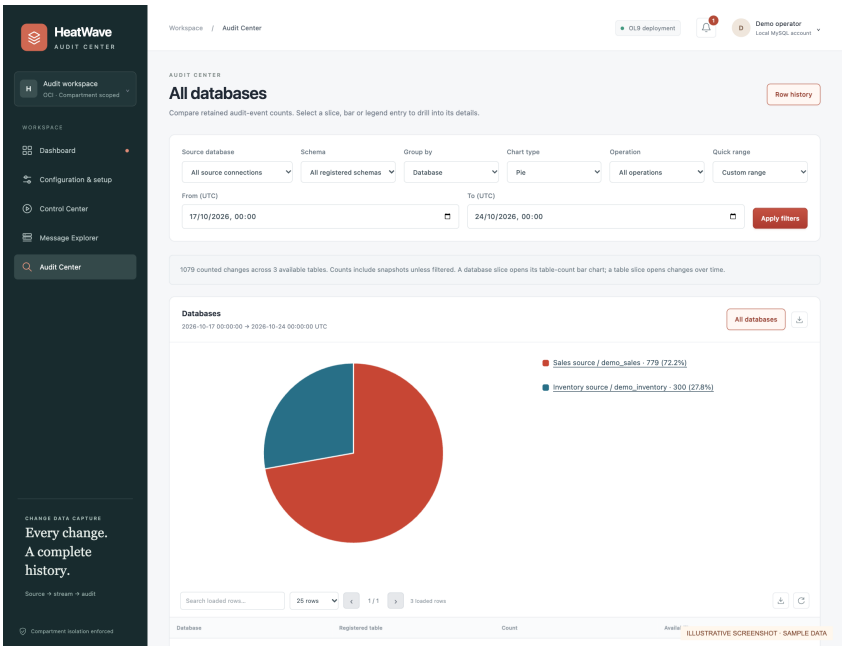
For DML, Debezium uses `c` (create), `u` (update), `d` (delete) and `r` (snapshot read). The target translates these to CREATE, UPDATE, DELETE and SNAPSHOT. Delete tombstones with null value are transport records, not a second business-row deletion.

This screen samples row data. Public schema events are archived separately and do not become ordinary row-image history entries. If the queue has the event but Audit Center does not, inspect writer health, target access, schema decoding, target checkpoint and any retention/conflict error. Do not move the source capture position to mask a downstream failure.

Oversized-event parts are labelled with their fragment index/count in Message Explorer. They are transport pieces; the audit writer creates an event only after complete validated reassembly.

USING THE APPLICATION

33. Audit Center: all databases overview



All databases pie chart and table availability. Illustrative screenshot using the current templates and isolated sample data.

Audit Center opens at the global overview. It counts retained target audit events across registered tables, not current business-row counts. Global views are limited to 100 tables with a 15-second counting budget; narrow the source selection for larger estates.

Field / action	Purpose
Source database	All source connections or one registered source
Schema	All registered schemas or one selected schema
Group by	Database or registered table
Chart type	Pie or bar for the global count comparison
Operation	All or one operation; snapshots count unless excluded
Quick range / From / To (UTC)	Preset or custom retained time window
Apply filters	Recount the requested view within bounded query budgets
Chart slice / legend link	Database → its table bars; table → its changes-over-time view
All databases	Return to the broad grouping
Download chart PNG icon	Export the displayed chart image

The companion table shows database, registered table, count and availability. An unavailable or unprepared table is excluded rather than counted as zero. If the notice reports exclusions or a count-budget limit, the displayed total is incomplete.

Use the same range/operation when comparing overview and drill-down counts. Count changes can reflect additional delivered events, retained-history expiry or changed filters. Snapshot initialization of one million rows can contribute one million SNAPSHOT records; it is not one summary event.

USING THE APPLICATION

34. Audit Center: table history and lookup

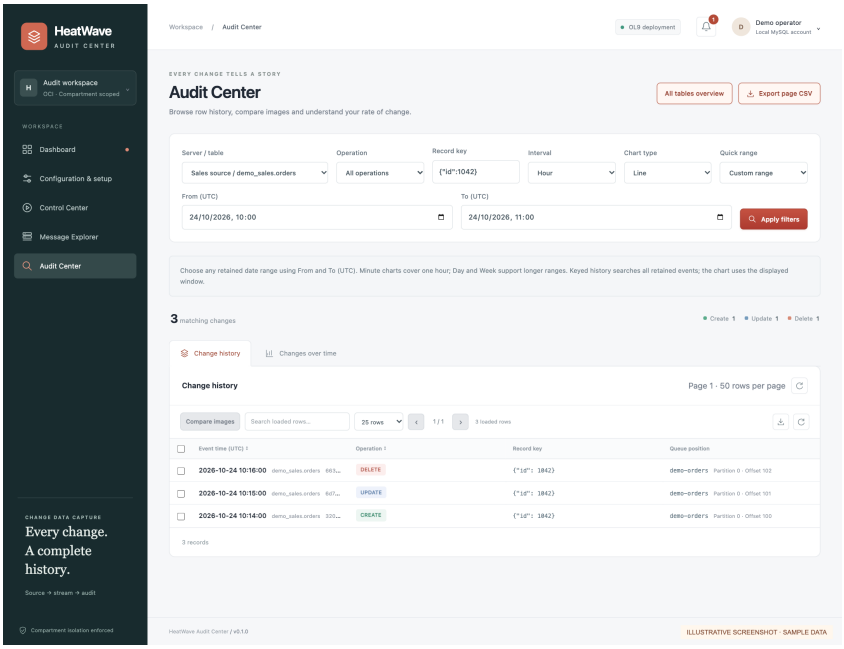


Table-specific history with a typed record-key lookup. Illustrative screenshot using the current templates and isolated sample data.

Choose a registered **Server / table**. Use **Change history** for individual events and **Changes over time** for charts. Tabs retain their selected view while applying filters.

Field / control	Purpose / condition
Server / table	Select the exact flow/source table; histories are scoped by registration/source identity
Operation	CREATE, UPDATE, DELETE, SNAPSHOT or TRUNCATE; empty means all
Record key	Full primary-key JSON object with original JSON types, for example {"id":1042}
Interval / Chart type	Time chart grouping and line/bar/stacked-bar presentation
Quick range	Fill a common range; custom From/To remains available
From / To (UTC)	Explicit retained range; both supplied bounds are inclusive in history queries
Apply filters	Run the requested target query and refresh results

Field / control	Purpose / condition
Export page CSV	Export the current loaded result page; not all retained history
Previous / Next	Server-side history pages of 50 events
All tables overview	Return to the global view

Unkeyed browsing defaults to a bounded 24-hour window. Keyed lookup without an explicit start can search retained history for that key; the chart still uses its displayed chart window. Supply a sensible range for predictable query cost and comparable counts. Shared toolbar search/sort acts on loaded rows only.

Each row shows event time/identity, operation, record key and queue topic/partition/offset. Select exactly one row and choose **Compare images**. Click a field value for its full text. A key typo, wrong JSON type, wrong table/source or expired range can produce an empty result without implying capture failed.

For wildcard flows, select the actual source database/table. Primary-key lookup needs one actual table; `*.*` also needs a database. Grouped destinations are combined for the selected flow.

USING THE APPLICATION

35. Before/after images and operation meanings



Selected UPDATE showing its before and after images. Illustrative screenshot using the current templates and isolated sample data.

Operation	Meaning	BEFORE / AFTER
CREATE	INSERT observed through the binlog	null / inserted row
UPDATE	Change to an existing source row	prior row / new row
DELETE	Source row removed	final row / null
SNAPSHOT	Existing row observed during initialization	null / current row at snapshot time
TRUNCATE	Table-level truncate evidence if delivered/supported	No reconstructed per-row deleted images

The Row change details dialog includes event identity, formatted images and an expandable **Source metadata** section. Use source coordinates alongside queue position to investigate delivery/replay. **× / Escape** closes the dialog.

SNAPSHOT is not an original INSERT history. With initial Start, an existing million-row table can produce a million snapshot records; its earlier changes are not reconstructed. With a genuinely new End flow there is no existing-row snapshot, but schema metadata is still captured. Existing offsets take precedence on normal restarts.

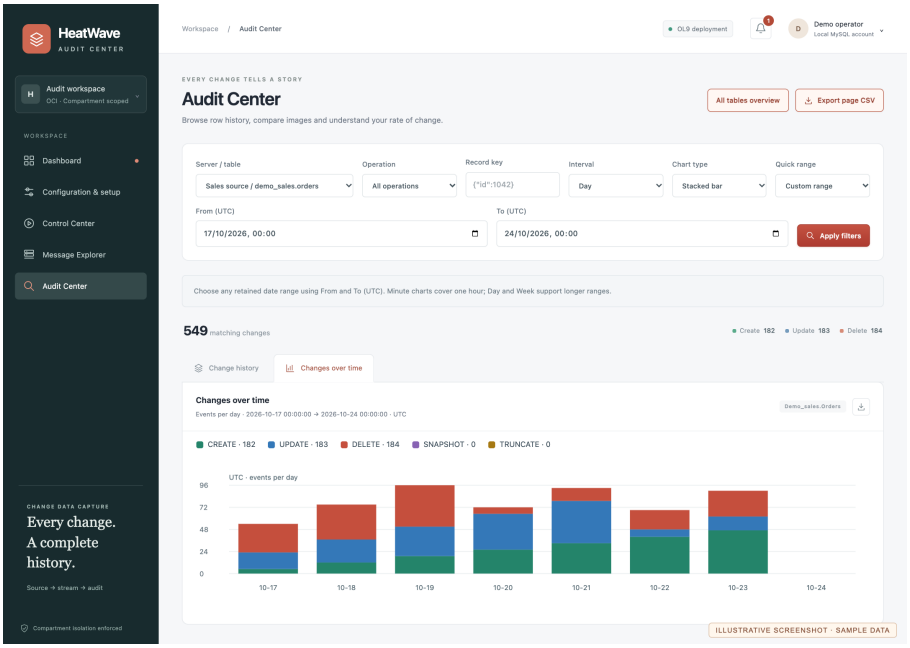
Full update/delete images depend on what the source recorded. FULL row images and complete JSON settings must be correct before the relevant transaction is logged. Decimal, date/time, binary and other MySQL values follow the connector's converter representation; review the stored schema/source metadata rather than assuming every JSON string is a source VARCHAR.

The target retains a deleted row's earlier evidence until an explicit eligible retention cleanup removes payload history. Source deletion does not delete target history. Row images alone do not identify every actor or original SQL statement.

DDL exception: bulk partition/table operations can remove or exchange source rows without individual CREATE/DELETE images. An old event is the last observed row version, not proof that the row still exists. No DELETE event does not mean no deletion; see Appendix I.

USING THE APPLICATION

36. Charts, intervals and downloads



Operation legend and custom-range stacked-bar chart. Illustrative screenshot using the current templates and isolated sample data.

Use **Changes over time** after selecting a table. The operation legend explains the series; do not interpret two lines without their labels.

Setting	Intended use
Minute (one-hour view)	Inspect minute buckets within one hour
Hour	Compare hourly activity in a bounded range
Day	Examine longer custom ranges by UTC day
Week (Monday UTC)	Aggregate longer periods into Monday-based UTC weeks
Line	Compare the direction and timing of operation rates/counts
Bar	Compare event counts per bucket
Stacked bar	Compare total volume and its operation composition

The chart shows event counts per selected interval, not a guaranteed records-per-second capacity measurement. Change Interval and From/To together when expanding the range; the app bounds the chart to 1,000 buckets and query size. Minute mode is intentionally a one-hour view.

Use **Quick range** for last hour/24 hours/7 days/30 days/90 days, or enter custom UTC dates. **Apply filters** reruns the query. Empty buckets and no-data notices must be distinguished from unavailable target queries. Counts include SNAPSHOT unless the operation filter excludes it.

The chart download icon saves the displayed PNG. **Export page CSV** exports history rows on the loaded page. The global overview offers pie/bar comparisons and drill-down to this time view. Keep range, table and operation consistent when sharing screenshots or comparing charts.

DDL-driven row-set changes are not counted as individual DML unless row events were actually emitted. A large partition exchange/drop can have no row-volume spike. These charts measure captured audit events, not every source data change.

USING THE APPLICATION

37. Schema changes and security auditing

DDL: structural evidence

The public schema stream carries available CREATE/ALTER/DROP and snapshot schema observations. The writer stores `audit_schema_changes` separately from row payloads: prior definition, new definition, semantic delta, DDL text, source position and fidelity. A schema baseline records what was known at initialization; it is not proof that an earlier CREATE was captured.

For example, adding `region VARCHAR(16) DEFAULT 'EMEA'` and widening `amount DECIMAL(10,2)` to `DECIMAL(14,2)` can produce column-addition and precision-change differences. Compare properties actually present in the definitions. Defaults, indexes and partitioning can remain unknown/partial. An ALTER does not imply a new UPDATE audit event for every existing row.

There is no dedicated DDL comparison page in the current app. Use an authorized SQL client for schema evidence:

```
SELECT observed_at, object_name, change_kind, fidelity,
       before_definition, after_definition, semantic_delta
FROM demo_audit.audit_schema_changes
WHERE flow_id = 1 AND object_name = 'demo_sales.orders'
ORDER BY observed_at, event_id;
```

Replace the example schema/object/flow with the intended registration. Retain private Debezium recovery history independently; it is required to decode later events and is not a disposable presentation cache.

DCL: user and privilege evidence

CREATE USER, GRANT and REVOKE are security/account-management operations. This prototype does not collect them as a complete security history or expose a DCL browser. Do not infer a privilege timeline by auditing changes to an incidental MySQL audit-configuration table.

An extension requires MySQL Enterprise Audit filters/user assignments and a separate collector that retains actor, account, privilege/scope, action, outcome and time. Oracle documents that audit logging is disabled by default until filters are configured. This collector, its target model and completeness validation remain future work. MySQL Enterprise Audit guidance.

DDL can change the row set without row images

EXCHANGE PARTITION can swap millions of rows without changing the column structure; DROP/TRUNCATE PARTITION can remove rows without individual DELETE images. ROW/FULL does not create the missing events. An empty semantic delta is not proof of unchanged data. The current schema writer requires matching decoded tableChanges entries and is not a raw-DDL catch-all. See Appendix I for coverage and mitigations.

OPERATION AND MAINTENANCE

38. Daily operations and a controlled verification

Start-of-shift checks

1. Review Dashboard attention counts, then each affected Control Center process/task tip.
2. Confirm latest source-to-queue-to-target progress with a known event or authorized monitoring/checkpoint queries. Running status alone is insufficient.
3. Review source binlog age/storage, queue retention/lag and target partition runway/storage. Alerting/lag autoscaling is not provided automatically by this UI.
4. Review failed builds, safe OCI request IDs and operational notifications. Preserve diagnostic evidence before clearing messages.

Authorized test-table example

Use a disposable registered test table, not an arbitrary production table. With initial End already established and healthy workloads:

```
INSERT INTO demo_sales.orders (id,status,amount)
VALUES (1042,'new',80.00);
UPDATE demo_sales.orders SET status='paid',amount=99.95
WHERE id=1042;
DELETE FROM demo_sales.orders WHERE id=1042;
```

In Message Explorer, look for row envelopes/coordinates. In Audit Center select the table and key `{"id":1042}` with the correct UTC range. Expect CREATE (after only), UPDATE (both images) and DELETE (before only). Compare values and target checkpoint progress. Reuse a key only if its previous history is understood.

For a planned pause, use Stop to pause capture while draining the queue. For target maintenance/cleanup use Stop both workloads and wait for INACTIVE. Resume within retention, then verify evidence and backlog recovery. Do not silently reset an expired queue offset or erase a conflicting record to make a process run.

Retiring a flow stops workloads and retains audit evidence. Terminate retired containers and delete the registration only after reviewing dependencies/recovery. OCI resource deletion is a separate guarded lifecycle operation.

OPERATION AND MAINTENANCE

39. Summary and handover checklist

The application coordinates a registered table's journey from HeatWave binlog through Debezium and OCI messaging to retained audit evidence. Its useful result is a traceable before/after history that can be retrieved by typed row key and explored over time.

Before handing over an evaluation deployment, record:

- Its prototype status, intended evaluation scope and accepted capability limits.
- The app owner, OCI/database administrators, VM compartment, region and approved network access.
- Source/target registrations, complete source offsets and private JDBC history, both public streams and image release/digest selections.

- The stable fingerprint-secret reference and the private state that must be preserved for recovery.
- Evidence of full-image capture, process/task health, target-before-ack behavior and row-key lookup.
- Tested source/queue outage windows, expected peak rates, target capacity and monitoring responsibilities.
- Explicit retention/hold decisions, backup/restore plans, cleanup approval ownership and rollback boundaries.

Keep real deployment inventories/runbooks under private, Git-ignored `internal_doc/`; the shareable guide uses examples only. Refer to the appendices for scaling, controlled configuration changes, updates and retention cleanup.

This remains an independent prototype for evaluation and rehearsal, not an Oracle-supported product. A successful demo or unit-test run does not establish production readiness, complete security auditing or a throughput/service guarantee.

APPENDICES

Appendix A. Scaling and massive change volume

One flow has one embedded Debezium reader and one history writer. A flow can capture one table, a database wildcard or an all-database wildcard. Grouping audit targets by flow/database/table does not add a reader or writer per table.

One MySQL binlog reader is not parallelized by increasing connector task count. Independent overlapping readers can multiply source/network work; scope overlap checks prevent unsupported duplicate mappings. The target writer lease enforces one active writer per flow. Arbitrary horizontal replicas are not supported.

Observed bottleneck	Review
Source ahead of capture	Source/binlog retention, reader CPU/memory and serialization size
Stream ahead of target	Writer batches, target commits/indexes, conflicts and fragment custody
Streaming 429 or per-entry rejection	Bytes per partition, hot keys, batch size and PUT/GET pacing
Container lifecycle/log API 429	Management polling/operation rate; separate from Streaming traffic
Large full-row updates	Both images plus full schema, logical limit, target packets and memory
Slow broad browsing	Narrow source/key/time filters; selected JSON page volume still matters

The writer's ordinary defaults remain 250 records, 4 MiB and 500 ms; one complete logical event may exceed the ordinary batch byte budget within its logical cap. Publishing batches up to 500 records and 900,000 estimated encoded bytes by default. GETs fetch up to 1,000 records and back off on idle queues. Tune only after measuring request counters, byte volume, delivery lag, target latency and peak/catch-up load.

Plan partitions and retention in advance. Changing stream identity requires an explicit checkpoint/lineage migration. Automatic scaling, target sharding, preaggregated chart statistics and compact schema-free envelopes are not implemented. Grouped target automatic retention and deployed wildcard historical repositioning remain disabled. See Streaming and size limits.

APPENDICES

Appendix B. Changing configuration safely

Intended change	Supported route	Continuity consideration
Rename a registration	Select one row → Edit	Keeps IDs, connector namespace, group and fingerprints
Change a draft mapping	Edit before deployment history	Revalidate live objects, queue and target compatibility
Change source/target endpoint of a deployed flow	New reviewed registration/migration	Name-only Edit protects existing identity; history adoption is not automatic
Choose a new binlog position	Capture start preview/apply	Requires retained schema/log coverage; can replay/skip changes
Select a new image	Select one flow → matching Update image action → release dropdown	Choice applies only to that flow; defaults and other flows stay unchanged
Change batch/TLS/runtime settings	Runtime settings revision plus controlled workload refresh	Existing instances retain their effective revision; test compatibility
Rotate database password	Coordinated Vault value rotation and workload refresh	Keep the supported secret reference; avoid source/writer login outages
Rotate row-fingerprint key	Not implemented for existing history	Preserve both key value and reference; changing value breaks lookup continuity
Change stream/pool/incarnation	Explicit migration/recovery plan	Never silently adopt new offsets or reset expired positions
Enable payload expiry	Per-flow retention policy plus stopped-flow cleanup	Durable cutoff persists; metadata/identity state is retained

Back up the current configuration and recovery state before changes. Record the intended scope, operator, old/new revision and validation. Keep region/compartment fixed. When the UI locks a field, do not bypass it with direct repository edits; the lock preserves identities and checkpoints.

For a runtime change without an image change, review the actual replacement path first. The current Update submission requires a release different from the deployed image; saving JSON alone is not a generic rollout controller. Build/select a reviewed release when needed to produce a controlled replacement carrying the new effective settings.

After a change, check process/task health, queue arrivals, target checkpoint and a known row-history query. A green configuration-save message confirms the save, not successful deployment or data delivery.

APPENDICES

Appendix C. Update the application and workloads

Web control-plane update on OL9

1. Back up the embedded repository, private configuration/runtime selection, installation identity, TLS material and relevant target recovery evidence. Coordinate a maintenance window and confirm no image build is active.
2. Verify the checkout remote/branch and clean working tree. Review the intended commit and dependency/schema changes before obtaining it. Never overwrite private state to make the tree clean.
3. Fetch the approved change, then update with a fast-forward pull when appropriate. Review differences before running setup.

```
git remote -v
git status --short
git fetch origin
git log --oneline HEAD..origin/main
git pull --ff-only origin main
./setup.sh
```

Setup reuses saved runtime defaults, validates units/dependencies and restarts the web service; it can interrupt web sessions. It does not reset an existing administrator or upgrade embedded MySQL across major versions. An Admin auto-update screen is not implemented in this application.

Debezium/history-writer update

Build and publish the changed component from the updated VM checkout. Select one flow in Control Center and use its matching Update image action. In the dialog choose the completed component release and review its build time, repository and digest before submitting. This selection applies only to that flow; it does not save a new global default. Preflight runs before stopping; replacement waits for confirmed INACTIVE, preserving supported identities/checkpoints. The other component retains its current state.

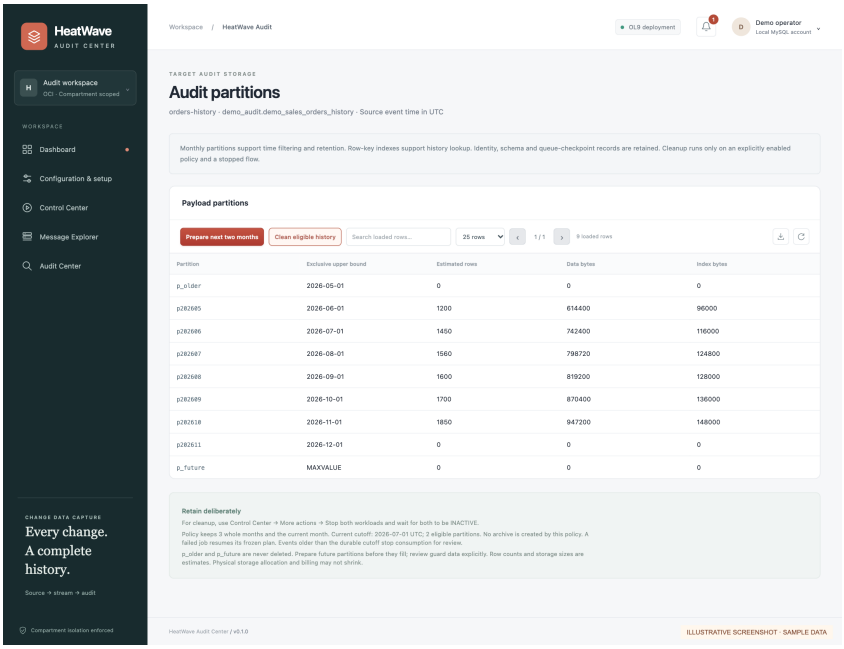
A writer update can grow the backlog while Debezium continues. A connector update pauses capture while source binlogs continue. Ensure retention covers the interruption. If shutdown takes too long, wait, reopen Update image and choose the intended release again; do not create a parallel unmanaged instance.

Prepare required target layouts using the reviewed application path or `.venv/bin/python scripts/prepare_targets.py --flow-id <id>`. Incompatible existing payload tables are refused rather than silently migrated. Review schema-history reconciliation and protocol compatibility before replaying old evidence through new code.

Verify a controlled source event, target images and checkpoint progress after replacement. Rolling back an image is not a rollback of target DDL, a cleanup cutoff or source transactions; each needs its own recovery review.

APPENDICES

Appendix D. One-action eligible payload cleanup



Audit partitions with an explicit retention policy and eligible-history action. Illustrative screenshot using the current templates and isolated sample data.

Cleanup is opt-in, per flow, destructive and deliberately scoped to eligible monthly payload partitions. It is not automatic deletion of all data or a full archive facility.

In Runtime settings, use the actual flow ID under top-level `retention`:

```
{"retention":{"1":{"enabled":true,"retain_months":3,
 "archive_required":false,"legal_hold":false}}}
```

This example keeps three whole UTC calendar months plus the current month. Both archive and hold decisions must be explicitly false. A legal hold or archive-required policy blocks cleanup; verified archive export/restore is not implemented.

1. Review the retention/hold decision and any required external preservation. Save the policy.
2. In Control Center use **More actions** → **Stop both workloads** and wait for both instances to be INACTIVE. The backend independently checks this state and ownership.
3. Open **Audit partitions**. Review the cutoff, eligible partition list and estimates. **Prepare next two months** extends future runway; it does not expire old history.
4. Choose **Clean eligible history** and confirm the irreversible deletion. One action runs the bounded job and records durable progress. A failed job resumes its frozen plan rather than widening the cutoff on retry.
5. Check Cleanup jobs and remaining partitions. Restart only after reviewing the cutoff and any late/replayed events.

The action retains `p_older`, `p_future`, identity ledger, schema state, delivery checkpoints, retention state and job manifests. It drops only eligible named monthly partitions, with a maximum of 24 per job. Guard data needs separate investigation. The durable cutoff stops ingestion of older events for review, preventing expired payload from silently returning through replay.

The ledger and metadata continue growing. Their deletion needs a validated replay-floor/archive policy and is not coupled to this button. Physical storage allocation or billing may not shrink after dropping partitions.

Source partition DDL and target housekeeping are different. This target cleanup intentionally expires audit payloads and records cleanup/cutoff state; it does not generate one DELETE audit event per expired payload and is not an archive.

Automatic cleanup is disabled for database/table grouped targets. Coordinated payload, identity, routing, delivery and fragment maintenance requires separate review.

APPENDICES

Appendix E. Target data layout and retrieval

The target schema contains the per-flow row payload table plus shared evidence/recovery tables. These are distinct from the embedded control repository.

Store	Purpose / retained evidence
Selected payload audit table	event_time/event_id, flow/source/table, operation, complete key, key hash, images and fingerprints, source metadata, schema document, queue coordinates
audit_event_dedup	Unique event identity and payload fingerprint; protects supported replay across partition boundaries
audit_schema_changes	Public DDL/schema evidence: before/after definitions, semantic delta, source position, fidelity
Per-flow private JDBC schema-history table	Debezium recovery/decode state for OCI Streaming
audit_native_offsets	Full native source offset state, including GTIDs and within-transaction progress
audit_target_routes / claims	Source namespace to owned physical audit destination
audit_message_assemblies / fragments	Durable incomplete custody and compact completed replay receipts
audit_delivery_checkpoints	Durable target next offsets by delivery partition/incarnation
audit_capture_requests	Reviewed capture-position requests and execution evidence
audit_retention_state	Monotonic payload expiry cutoff and target identity
audit_cleanup_jobs / manifests	Frozen cleanup plan and durable progress

New row payload tables use monthly RANGE COLUMNS(event_time) partitions, primary key `(event_time,event_id)` and a row-key history index. The unpartitioned identity ledger provides cross-partition identity uniqueness. Guard partitions contain older/out-of-runway events and are not ordinary expiry candidates. Existing unpartitioned/incompatible tables require an explicit migration.

The browser receives a typed key, computes the same HMAC as the writer and queries the row-key/time index. Event time supports partition pruning; a bounded range reduces work. Sorting/paging loaded rows is different from server query planning. Large broad searches and global count aggregation remain bounded and are not

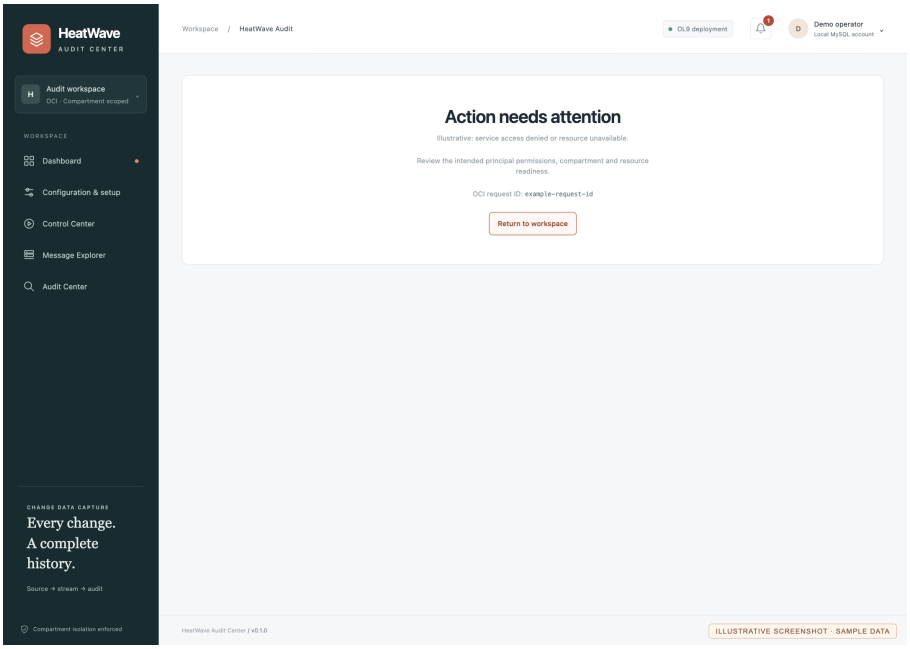
preaggregated analytics.

The target is audit evidence, not a current-row replica: a DELETE has a retained before image, a SNAPSHOT is an observation and a DDL event belongs to structural history. Schema fidelity may be partial, and cross-topic arrival order does not establish one global transaction order across tables/databases.

Use before/after capture and retrieval for field-level detail and design validation for checkpoints, partition and migration contracts.

APPENDICES

Appendix F. Troubleshooting by observed stage



Action needs attention with safe guidance and a request identifier. Illustrative screenshot using the current templates and isolated sample data.

Symptom	Inspect / next action
Connection test fails	Host/port reachability, database/account/grants, Vault reference/value, VM secret access and global TLS policy
Source checks require RELOAD/flush access	Dedicated account grants/roles; supported HeatWave FLUSH_TABLES alternative with LOCK TABLES
Fingerprint key missing	Runtime settings → Generate and configure key using permitted vault/key
OCI inventory 403/404	Correct principal group/domain, service-specific permissions, compartment, region and resource readiness; retain request ID
Stream creation result ambiguous	Refresh/reconcile the pool/name/app provenance before retrying; never blindly duplicate
Deploy requests both topics	Prepare data and single-partition public schema stream; verify the actual stream identities and delivery checkpoints

Symptom	Inspect / next action
Debezium running, task unavailable	Fresh runtime/task health and log permissions, image compatibility, source/secret/network settings
Writer inactive or exited	Start existing healthy image, or select fixed release and Update writer; inspect target/checkpoint/decode errors
Queue has event, target has none	Writer state, target transaction/permissions, schema/type parsing, expiry cutoff or identity conflict
No source movement with End	New transactions after initialization, committed changes, correct table/database, retained binlog coverage
Image update disabled	Select exactly one non-retired flow whose relevant deployed instance is ACTIVE/INACTIVE; the dialog requires a different available component release
Capture-position request rejected	Binlog/schema baseline retained, executed set cumulative, compatible protocol, coverage not ahead of durable decoded history
Audit lookup empty	Correct source/table, complete typed key, UTC range and retained payload; verify source/queue/target stages
Cleanup unavailable/blocked	Enabled explicit policy, no hold/archive requirement, both workloads stopped, protocol/layout/ownership checks
HTTP 412 during DB lifecycle	Refresh lifecycle/precondition state; review safe service error/request ID before retry

Use Notifications to retrieve recent messages; clearing them is not a remedy. The Action needs attention page gives the safe operation message, optional guidance/policy example and request ID. Use its return action to revisit the appropriate workspace.

Do not reset offsets, drop ledgers, replace the fingerprint key or delete evidence to force a successful status. A poison record/conflict stops consumption for investigation and is not silently skipped. Preserve the first error, effective image/config revision, flow identity, timing and relevant coordinates without copying passwords or row images into public tickets.

For Streaming 429, identify PutMessages/GetMessages and inspect bytes, batch counts, hot partitions and paced request deltas. HTTP 200 can contain rejected PUT entries. Container lifecycle/log API 429 is a separate management quota. Incomplete fragment custody and target packet failures need evidence review; repeatedly restarting is not a capacity solution.

APPENDICES

Appendix G. Security, backup and recovery boundaries

Security responsibilities

Use least-permitted same-compartment principals, private worker/DB paths, restricted web ingress and dedicated DB accounts. The default MySQL TLS mode encrypts without certificate verification; enable reviewed global verified trust when required. Authentication, CSRF, server-side sessions, scope and ownership checks protect app actions, but do not replace deployment security review.

Audit payloads can contain the full source row, including personal or sensitive values. Restrict access to target history, screenshots, exports, backups and logs. Redaction/column filtering and a complete field-level access model are not implemented. A history fingerprint is not encryption, anonymization or proof against all tampering.

Preserve the state that defines continuity

Back up the embedded repository consistently, private deployment JSON, saved runtime selection, app installation identity and TLS material. Back up target payload/ledger/schema/checkpoint/capture/retention state together according to the chosen recovery objectives. Preserve the fingerprint secret and source identity. Retain required binlogs and queues; a backup does not make purged source/queue messages available again.

Use administrator-approved database/OCI backup tooling; the app has no one-click full backup/restore or verified archive facility. Keep backups private with appropriate encryption/access and test restoration on an isolated deployment. Do not publish them with this manual.

Failure recovery rules

Normal Start/Resume uses durable checkpoints. The writer checks saved next offsets against queue watermarks and fails closed when history has expired or an offset is beyond queue end. Recreating a queue requires a new incarnation and reviewed recovery plan; silently seeking earliest/latest can lose evidence.

A replay can revalidate supported event identities; a mismatched payload for an existing identity is a conflict, not an overwrite. Snapshot observations differ from original DML. Source failover/server-identity changes, target restoration and connector lineage changes need explicit acceptance testing.

Do not claim a zero-loss recovery point or global transaction reconstruction until the actual chosen services, backups, retention and replay boundaries have been tested. Record any coverage gap as part of the audit evidence.

APPENDICES

Appendix H. Glossary and further reading

Term	Meaning
Debezium Engine	Embedded MySQL binlog reader producing typed change/schema events
Source scope	One table, `database.*` or `*.*` after exclusions
Target grouping	One audit destination per flow, source database or source table
GTID set	UUID/interval groups describing transactions already processed
Purged GTIDs	Executed transactions no longer available in retained binlogs
Native source offset	Full file/position/event/row and GTID recovery state
Stream identity	Actual OCI stream ID binding delivery state to its retained data
Fragment custody	Durable saved parts awaiting complete validated reassembly
Delivery checkpoint	Next stream offset after durable custody/audit evidence
Fingerprint	Keyed canonical identity/content hash; not proof of capture completeness

Further reading: Debezium MySQL, Debezium Engine, OCI publishing, OCI throttling, MySQL JSON, capture positioning, source scopes, buffering and size limits.

APPENDICES

Appendix I. DDL-driven row changes and audit coverage

This prototype cannot promise complete row history for DDL-driven data changes. MySQL logs DDL as statements even in ROW mode. FULL supplies columns in row events; it does not create individual row images for a partition swap or drop. Debezium and the writer cannot recover missing values from a DDL statement.

Operation	What can happen	Evidence limitation
EXCHANGE PARTITION ... WITH TABLE	Swap a partition with an unpartitioned table	No individual CREATE/DELETE for the exchange, even if both objects are registered
DROP PARTITION	Usual INPLACE RANGE/LIST operation removes the partition and its data	No per-row DELETE/BEFORE records
TRUNCATE PARTITION	Empty chosen partitions without changing their definitions	No per-row deleted images; structural delta can be empty
TRUNCATE TABLE / DROP TABLE	Remove current table contents	A table-level/schema event is not a list of deleted-row images

Operation	What can happen	Evidence limitation
Column changes / defaults	Alter existing logical values or discard a column	No guaranteed UPDATE for every existing row

Exact effects depend on the supported MySQL version, engine and algorithm. DROP PARTITION with COPY can redistribute some rows rather than delete the entire old row set. REORGANIZE commonly preserves logical data while changing placement; physical movement is not automatically a logical UPDATE.

Schema history is not a partition-row snapshot

The separate public schema path can archive supported matching tableChanges, including available structure/DDL/source positions. It does not retain every partition row. The current writer has no raw-DDL catch-all; parser/version/filter behavior or no matching tableChanges can leave no schema record. Validate each exact statement on a test flow.

EXCHANGE can change millions of visible rows while retaining the same structure. An empty semantic_delta therefore does not prove unchanged contents. Client affected-row counts and before/after structure definitions are not row-set comparisons.

Effect on lookup, fingerprints and charts

A row removed through partition DDL can have an old CREATE/UPDATE and no later DELETE. A row brought in by exchange can have no CREATE. The most recent retained event is a last observation, not proof of current membership. Charts count captured events; a large DDL-driven change may not create a row-volume spike. HMAC fingerprints identify observed keys/values and do not prove capture completeness.

Restart, GTID repositioning, longer retention and ROW/FULL cannot synthesize absent events. A later snapshot is a new observation, not reconstruction of unavailable BEFORE values or intermediate changes. Do not label inferred differences as original Debezium DML.

Mitigation policy; not automated here

1. Have the DBA review/limit data-impacting DDL on tables requiring complete row history. The prototype does not automatically block or flag every such operation.
2. Where feasible and approved, use ordinary logged DML with capacity/locking/retention review for removals or imports requiring individual row evidence.
3. For unavoidable DDL, preserve complete affected keys/rows and definitions before execution, including both sides of an exchange, at a coordinated source boundary.
4. Preserve the statement/outcome/coordinates and a consistent after-state. Control concurrent writes with a DBA-approved procedure; counts/checksums alone do not recover row values.
5. Retain separately derived reconciliation evidence and any coverage gap. Pre/post materialization, snapshot comparison and derived-event storage are not implemented features.

A separate audit log can identify a statement/actor/outcome but not every removed row. Preserved exchanged tables/backups may help recovery; a DDL-only log cannot restore values absent from all retained evidence.

Sources: MySQL binary-log formats, partition exchange, partition drop/algorithm semantics, TRUNCATE TABLE. See the full limitations guide for additional snapshot, retention, DCL, key and scaling boundaries.

APPENDICES

Appendix J. HeatWave ETag issue and process errors

What failed and what changed

DB-system Start returned HTTP 412 NoEtagMatch even after fetching a fresh resource ETag. Some compressed GET responses used an ETag ending in --gzip. An uncompressed GET removed that representation suffix, but a subsequent resource-version mismatch still occurred. Compression was not the only observed cause; refreshing was not a sufficient remedy.

HeatWave OCI operations now omit ETag/If-Match. Oracle documents the lifecycle header as optional. The app keeps VM instance-principal authentication, exact compartment checks, required lifecycle state, request IDs and per-operation idempotency tokens. Deletion still requires app ownership and unused dependencies. It does not strip a suffix from an opaque ETag or change IAM to work around 412.

The read and action are not an atomic version lock. A concurrent operation can change state and OCI can still reject a request. A remaining 412 needs the requested action, resource name, last checked state, active work request and request ID for investigation. It is not automatically a password or IAM problem.

Why a writer can exit while its instance is ACTIVE

A stopped/unreachable target can cause MySQL 2003 during target connection or a lost-connection error during processing. OCI Container Instance lifecycle ACTIVE does not prove the application process is running. Debezium can independently retry a stopped source while its task still reports RUNNING.

Restore source/target DB availability, verify the configured connection and restart/resume the registered workload with existing checkpoints. Do not start every old INACTIVE instance; older stopped image versions can remain after replacements. Do not reset offsets, recreate streams or delete history to clear an attention badge.

What the latest-error display means

Each process block retains state/exit status and displays a compact latest safe logged error with time and relevant MySQL code. Clicking the process field opens its full diagnostic, including the failed step and next action. Examples include target connect, envelope decode, target/schema commit, checkpoint save and queue acknowledgement. Known codes are mapped to fixed guidance; raw exceptions, SQL, passwords and row values are not exposed.

Current images produce structured diagnostics. Older images may contain only an exception class, and unavailable log access is not proof that no error occurred. The display reports only evidence actually present and can show a historical error after a process recovers. Current health, queue arrival and target checkpoint progress remain separate checks.

OCI lifecycle and ETag notes · Oracle DB system SDK.

APPENDICES

Appendix K. Same-compartment resource limitation

HeatWave Audit Center is an independent prototype for evaluation and rehearsal, not an Oracle product. This installation supports **one VM compartment and the VM region**. OCI management calls use the VM instance principal. Private runtime configuration must match the compartment and region reported by the VM metadata service; selecting another compartment or switching credentials is not a bypass.

This is an **application design restriction**. It is not a statement that OCI services universally prohibit cross-compartment architectures. Shared resources in another compartment are outside this prototype's supported deployment model, even when an IAM policy could permit access.

What must be in the VM compartment

Resource or reference	Current application boundary
Administrator password secret	Select an ACTIVE secret from the VM compartment in Add connection; the app tests the selected credential before use
Dedicated Debezium/history-writer password secret	Created in the VM compartment; its username/reference replace the temporary administrator inputs
Vault and encryption key	Choose an ACTIVE Vault and enabled AES key in the VM compartment; the key must belong to the selected Vault
Existing credential secrets	Use a local-compartment secret and Vault; importing an out-of-compartment OCID is not a supported alternative to selection
Row-fingerprint secret	Dedicated, stable secret in the same compartment; preserve it for all existing history
OCI Streaming	Stream pool and data/schema streams in the same compartment
Container Registry	Private repository and selected images in the same compartment
Container Instances	Workloads and their selected subnet/VCN attachments in the same compartment
Managed networking	Existing VCNs, subnets and referenced NSGs checked by the provisioning path must be in the VM compartment; shared networking is not modified
OCI-managed HeatWave DB resources	List/create/start/stop/delete paths are compartment constrained; deletion additionally requires app ownership and dependency checks

The secret dropdown lists metadata and names, never plaintext secret values. Empty lists or denied metadata access do not authorize selecting from a different compartment. Restore the intended VM principal access or create the resource in the VM compartment. Check which principal needs access: the VM manages/tests resources, while containers independently resolve credentials and pull images using their resource principals.

Selecting a compartment-local secret is not a relocation of its existing Vault. When provisioning existing/shared resources manually, keep both the secret and its Vault/key in the selected boundary. New account/fingerprint setup explicitly checks the selected Vault/key attachments. The prototype does not implement a cross-compartment credential-sharing or secret-migration workflow.

Database addresses are a separate limitation

Connection registration accepts a database hostname/IP, port, database, username and selected Vault reference. It tests actual MySQL connectivity/login over TLS. It does not require a DB-system OCID or infer resource ownership from the address. **A successful host-only connection test does not prove the database's OCI compartment.** Operators must place the source/target DB systems in the intended compartment and verify their placement independently. The OCI HeatWave resource controls perform resource-ID compartment checks, but those checks do not turn an unassociated host-only registration into an attested DB-system identity.

Catalog and IAM distinctions

Shapes, service versions and Oracle-owned default configuration catalogs are service-supplied choices, not resources created by this app in another customer compartment. Selecting an Oracle default configuration omits a foreign custom configuration reference. Tenancy-level IAM groups, dynamic groups and policies must be prepared by an administrator; the application does not create or change them.

Troubleshooting and future scope

1. Confirm the VM compartment/region and configured principal before investigating an empty resource dropdown or OCI 403/404.
2. Confirm the resource is ACTIVE/available and located in that compartment. Credential lookup requires secret metadata and secret-bundle access; creation additionally needs permitted Vault/key operations.
3. Check the VM and worker principal permissions separately. Do not broaden access or switch to API-key credentials to avoid the application boundary.
4. Preserve fingerprints, offsets, checkpoints, identity tags and secret values during any planned migration. The app does not automatically move/adopt shared resources or rewrite existing audit history.

Supporting multiple resource compartments or regions would require an explicit scope model, separately approved IAM/network access, attachment validation, credential handling and migration/recovery tests. Editing an OCID in private JSON alone is not that implementation.